TÉCNICO
UNIVERSIDADE
DE LISBOA



DEPT: Defense via Ensemble and Purification Transformer

Guilherme Silvério de Carvalho Romeiro Leitão

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Fernando Manuel Valente Ramos

October 2026

Declaration
I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetuer erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you – Thank you.

Abstract

Network intrusion detection today relies on machine learning, which brought a weakness signatures never had: the detector itself can be evaded. An attacker that reshapes only packet timing and sizes, keeping every packet protocol-compliant, can make an ongoing attack look benign to a deployed detector. Much of the defensive literature measures robustness against perturbations no real packet sequence can produce. This thesis proposes DEPT (Defense via Ensemble and Purification Transformer), a streaming defense trained and evaluated on realizable adversarial traffic from TANTGen, a black-box generator built in a sibling thesis. DEPT turns traffic into a masked, bit-level representation at two temporal granularities; two architecturally different learners, a Random Forest and a Temporal Convolutional Network, fused per packet by an attention-based meta-learner, decide whether a flow was manipulated; flagged flows are corrected by a causal purification Transformer and realized back into sendable packets. Across ten traffic sources from three public datasets, each manipulated in three perturbation modes, DEPT gives a verdict from the first packet, where the fusion is the best model available, and reaches an end-of-flow AUROC of 0.933; the Random Forest alone is stronger from the second packet on, and the fusion's benefit is earliness and a higher cost to a gradient-guided attacker, which never evades the stack on 74.5% of flows against 45.5% for the sequence model alone. Judged through the independent detector each attack targeted, purification recovers 63% of the destroyed detection and cuts attack success from 21.1% to 11.6%, with 97.2% of reconstructed packets parsed cleanly by Wireshark's `tshark`. The evaluation reports the failures with equal prominence: on the size-perturbed reflection floods no model beats chance, and purifying non-evading traffic lowers detection rather than restoring it.

Keywords

Adversarial Machine Learning; Network Intrusion Detection; Evasion Attacks; Ensemble Learning; Adversarial Purification; Streaming Traffic Analysis

Resumo

A deteção de intrusões em rede assenta hoje em aprendizagem automática, trazendo uma fragilidade que as assinaturas não tinham: o próprio detetor pode ser enganado. Um atacante que altere apenas tempos e tamanhos de pacotes, sem nunca violar o protocolo, faz um ataque em curso passar por benigno ao detetor. Muita da literatura de defesa mede a robustez contra perturbações que nenhum tráfego real consegue produzir. Esta dissertação propõe o DEPT (Defense via Ensemble and Purification Transformer), uma defesa em streaming treinada e avaliada sobre tráfego adversarial realizável do TANTGen, um gerador black-box de uma dissertação irmã. O DEPT converte o tráfego numa representação mascarada e ao nível do bit, a duas granularidades temporais; uma Random Forest e uma Temporal Convolutional Network, de famílias distintas, fundidas a cada pacote por um modelo de atenção, decidem se o fluxo foi manipulado; os fluxos sinalizados são corrigidos por um Transformer causal de purificação e reescritos como pacotes enviáveis. Em dez fontes de tráfego de três conjuntos de dados públicos, manipuladas em três modos de perturbação, o DEPT dá um veredicto logo no primeiro pacote, onde a fusão é o melhor modelo disponível, e atinge uma AUROC final de 0,933; a Random Forest isolada é superior a partir do segundo pacote, e o benefício da fusão está na precocidade e no maior custo para um atacante guiado por gradiente, que nunca evade a deteção em 74,5% dos fluxos, contra 45,5% no modelo sequencial. Avaliada pelo detetor independente visado por cada ataque, a purificação recupera 63% da deteção destruída e reduz o sucesso do ataque de 21,1% para 11,6%, com 97,2% dos pacotes reconstruídos aceites sem erros pelo `tshark` do Wireshark. A avaliação dá igual destaque às falhas: nos floods de reflexão com perturbação de tamanho nenhum modelo supera os 50%, e purificar tráfego que não evadia baixa a deteção em vez de a restaurar.

Palavras Chave

Aprendizagem Automática Adversarial; Deteção de Intrusões em Rede; Ataques de Evasão; Métodos de Ensemble; Purificação de Tráfego; Análise de Tráfego em Tempo Real

Contents

1 Introduction **1**

1.1 Context and Motivation . . . 2

1.2 Problem Statement . . . 3

1.3 Work Objectives and Challenges . . . 4

1.4 Rationale and Contributions . . . 4

1.5 Organization of the Document . . . 6

2 Background **7**

2.1 Machine Learning for NIDS . . . 7

2.1.1 Learning Paradigms . . . 8

2.1.2 Feature Engineering and Representation . . . 9

2.1.3 Model Architectures Relevant to DEPT . . . 10

A – Random forests. . . . 10

B – One-dimensional and temporal convolutional networks. . . . 11

C – Transformers. . . . 11

2.2 Adversarial Machine Learning . . . 12

2.2.1 Threat Taxonomy . . . 12

2.2.2 Defense Paradigms . . . 14

2.2.3 Realizability and Protocol Constraints . . . 15

2.3 Ensemble Learning for Robustness . . . 16

2.3.1 Bagging, Boosting, Stacking . . . 16

2.3.2 Gradient Masking and Transferability . . . 16

2.4 Summary . . . 18

3 Related Work **19**

3.1 Adversarial Attacks against ML-based NIDS . . . 20

A – Feature-space attacks. . . . 20

B – Traffic-space attacks. . . . 20

3.1.1 TANTGen, the Attacker of This Thesis . . . 21

3.1.2 Gradient-guided Evasion, the Second Attacker . . . 21
3.2 Training-time Defenses . . . 22
A – Data sanitization and poisoning defenses. . . . 22
B – Federated and backdoor defenses. . . . 22
C – Ensemble-based hardening. . . . 23
D – Realizable adversarial training. . . . 23
3.3 Inference-time Defenses . . . 24
A – Certified robustness. . . . 24
B – Generative purification. . . . 24
C – Constraint-based and auxiliary detection. . . . 25
3.4 Positioning of DEPT and Gap Analysis . . . 25
4 Solution: DEPT Architecture **29**
4.1 Threat Model . . . 30
4.2 System Overview . . . 31
4.3 Dataset Construction and Packet-to-Tensor Pipeline . . . 33
A – Traffic corpus. . . . 33
B – Label convention. . . . 34
C – From packets to tensors. . . . 34
D – Splits and class balance. . . . 34
4.4 Primary Detection: Heterogeneous Stacking Ensemble . . . 35
4.4.1 Random Forest Base Learner . . . 35
4.4.2 Temporal Convolutional Network Base Learner . . . 36
4.4.3 Meta-Learner (the “Mini-Transformer”) . . . 36
4.5 Input Purification: Reconstructor . . . 37
4.5.1 Purification Model . . . 38
4.5.2 Training on Matched Flow Pairs . . . 39
4.5.3 Traffic-Space Realization and Validation . . . 39
4.6 Implementation and Tools . . . 40
5 Evaluation **43**
5.1 Research Questions . . . 44
5.2 Experimental Setup . . . 44
5.2.1 Datasets and Threat Generation . . . 44
5.2.2 Implementation Environment . . . 45
5.3 Metrics . . . 46
A – Static classification. . . . 46

B – Streaming. . . . 46
C – Purification. . . . 46
5.4 Base Learner Results . . . 47
5.5 Meta-Learner Results . . . 49
5.6 Purification Results . . . 51
5.7 Ensemble Robustness under a Gradient-guided Attacker (RQ1) . . . 55
5.7.1 The Attacker . . . 55
5.7.2 How the Numbers Should Be Read . . . 56
5.7.3 Results . . . 57
A – The stack costs the attacker more than either base learner alone, at every budget an attacker would plausibly pay. . . . 57
B – The meta-learner shifts its reliance onto the Random Forest as the TCN is degraded. . . . 57
C – Gradient guidance gives this attacker no aggregate advantage. . . 58
5.8 End-to-End System Evaluation . . . 59
5.8.1 Detect-then-Purify, End to End . . . 59
5.8.2 One Global Model or Ten Specialists? . . . 62
5.9 Discussion and Limitations . . . 64
A – The research questions. . . . 64
B – Purification without a good gate can subtract as well as add. . . . 64
C – The reflection floods' `size` variants fail below chance. . . . 64
D – A stacker beaten by its own base learner. . . . 64
5.9.1 Scope of the Gradient-ablation Result (RQ1) . . . 65
6 Conclusion **67**
6.1 Summary of Contributions . . . 67
6.2 Answers to Research Questions . . . 68
6.3 System Limitations . . . 70
6.4 Future Work . . . 71
Bibliography **73**

List of Figures

1.1 The adversarial evasion problem. 2

2.1 The two supervision regimes most relevant to DEPT. 8
2.2 The nPrint per-packet representation with identity masking. 10
2.3 The one-dimensional convolutional pipeline. 11
2.4 Dilated causal convolutions and the TCN receptive field. 12
2.5 The decoder-only Transformer block. 13
2.6 Purification as projection onto the benign manifold. 15
2.7 The three classical ensemble strategies. 17

3.1 The surveyed defenses on the phase and space axes. 26

4.1 DEPT's deployment context. 30
4.2 The DEPT architecture. 32
4.3 The dual prediction cadence over a flow. 35
4.4 The purification stage. 38

5.1 Per-source AUROC distribution. 49
5.2 Streaming AUROC@t trajectories. 50
5.3 Purification gain against perturbation footprint. 54
5.4 A single real packet through the purification pipeline. 55
5.5 Evasion cost: one perturbation, three targets. 58
5.6 Deployment baseline: NIDS attack success before vs. after DEPT. 60

List of Tables

2.1 The two representations DEPT derives from the same masked nPrint tokens. 10

3.1 Comparison of the surveyed adversarial defenses. 26

4.1 The 13-dimensional fusion token. 37
4.2 Model configuration summary. 41

5.1 Static classification results 47
5.2 Per-source streaming AUROC 48
5.3 Streaming comparison 50
5.4 Purification results (overall) 52
5.5 Purification by source 53
5.6 Evasion cost under a gradient-guided attacker. 57
5.7 Deployment baseline: attack success before vs. after DEPT 61
5.8 Global model against per-attack specialists 63

Acronyms

AML Adversarial Machine Learning

ART Adversarial Robustness Toolbox

ASR Attack Success Rate

AUROC Area Under the Receiver Operating Characteristic curve

CNN Convolutional Neural Network

DDoS Distributed Denial of Service

EDP Earliest Detection Packet

FGSM Fast Gradient Sign Method

FL Federated Learning

GAN Generative Adversarial Network

ICMP Internet Control Message Protocol

IFM Inverse Feature Mapping

IP Internet Protocol

IPAT Inter-Packet Arrival Time

MDP Markov Decision Process

ML Machine Learning

MSE Mean Squared Error

MTU Maximum Transmission Unit

NIDS Network Intrusion Detection System

PGD Projected Gradient Descent

RF Random Forest

RL Reinforcement Learning

SMT Satisfiability Modulo Theories

TCN Temporal Convolutional Network

TCP Transmission Control Protocol

TTL Time-to-Live

UDP User Datagram Protocol

1

Introduction

Contents

1.1 Context and Motivation 2

1.2 Problem Statement 3

1.3 Work Objectives and Challenges 4

1.4 Rationale and Contributions 4

1.5 Organization of the Document 6

Network intrusion detection has largely become a machine learning problem. The systems that guard today's networks rely less and less on hand-written signatures and more on learned models, and with that shift they inherited a weakness that signatures never had: the models themselves can be attacked. This thesis is concerned with that weakness in its most realistic form, and with what a defender can actually do about it. It proposes DEPT (Defense via Ensemble and Purification Transformer), a streaming defense that detects adversarially manipulated network traffic and undoes the manipulation before it reaches the detector it was meant to evade.

1.1 Context and Motivation

A Network Intrusion Detection System (NIDS) watches the traffic that crosses a network boundary and decides what is malicious. For a long time this decision was made by matching traffic against signatures of known threats, which is precise for catalogued attacks and blind to everything else. Machine Learning (ML) changed the terms of the problem: instead of enumerating what an attack looks like, a model learns the statistical behaviour that separates malicious from benign traffic, and can therefore generalize to variants and to attacks it never saw [1, 2]. Learning-based detection is by now the dominant research direction in the field and an increasingly common deployment choice [3, 4].

The improvement came with a new attack surface. A learned model can be fooled: a small, deliberate perturbation of its input may flip its decision, the phenomenon known as an adversarial example [5, 6]. For a NIDS, adversarial examples take the form of evasion: the attacker reshapes ongoing attack traffic just enough for the detector to label it benign, while the attack keeps doing its work [7]. The threat is not hypothetical. Black-box attacks have reached evasion rates of up to 35.7% against deep detectors that looked nearly perfect on clean benchmarks [8]. Figure 1.1 illustrates the situation this thesis starts from.

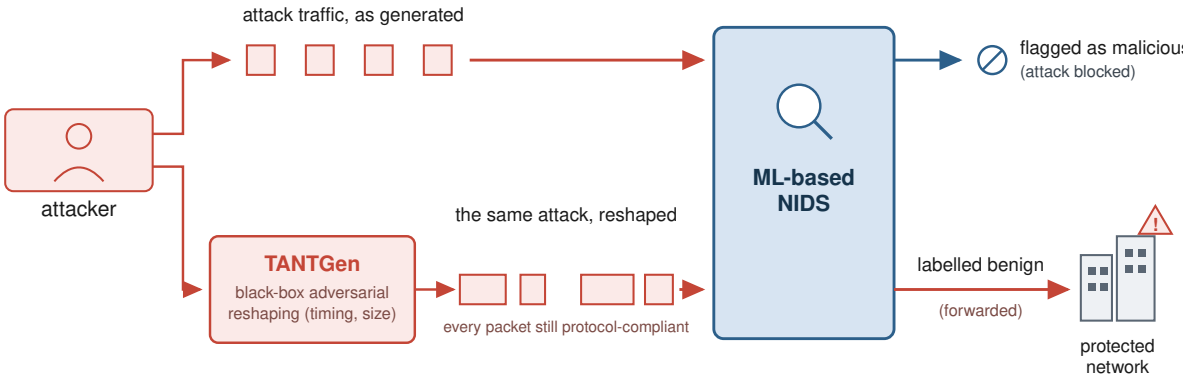


Figure 1.1: The problem this thesis addresses. In the form the attacker originally generates it, attack traffic is flagged by the ML-based NIDS that guards the network (top path). A black-box generator, TANTGen in this work, reshapes the same attack, altering only packet timing and sizes while every packet remains protocol-compliant, and the detector now labels the traffic benign, so the attack is delivered to the network it was aimed at (bottom path). DEPT exists to close exactly this gap.

Not every adversarial example in the literature is a real threat, however, and this point shapes the whole thesis. In the image domain, where adversarial examples were first studied, any perturbed pixel array is still an image; a perturbed feature vector of network traffic, in contrast, is only a threat if some sendable sequence of packets actually produces it. Recovering concrete traffic from a desired feature vector is the Inverse Feature Mapping (IFM) problem, and it is often intractable, since packets must keep their checksums, lengths, and protocol semantics consistent [9, 10]. A realistic attacker is then confined to the black-box setting: it can probe a deployed detector and observe outcomes, but it holds neither the model's architecture nor its gradients [11, 12]. Much of the defensive literature sets aside one or both

of these constraints, assuming white-box access or training against feature-space perturbations that no real packet sequence can produce, and so ends up measuring robustness against a threat that cannot occur as measured; Chapter 3 examines this mismatch in detail.

This mismatch is what the present work, together with its sibling thesis, set out to remove. TANTGen [13] closes the realism gap on the attack side: it is a Transformer-based generator that manipulates attack traffic under black-box constraints, altering only packet timing and packet size, and the traffic it emits is protocol-compliant by construction. DEPT is the defense side of the same effort. It is trained on, and evaluated against, the realizable adversarial traffic that TANTGen produces, so that every claim made about the defense refers to an attack that demonstrably exists on the wire, and not to a synthetic worst case.

1.2 Problem Statement

The problem this thesis addresses can now be stated. A stream of flows arrives at the boundary of a protected network, where a NIDS is deployed. Part of the traffic is benign; part belongs to conventional attacks, which the deployed NIDS is assumed to handle, since that is precisely its job; and part has been adversarially manipulated so that this NIDS mishandles it. The defense must decide, for each flow and while the flow is still in progress, whether it has been manipulated, and, when it has, neutralize the manipulation before the protected system consumes the flow.

Three constraints qualify an acceptable solution. First, streaming operation: the verdict must be produced packet by packet, as the flow unfolds, because a defense that waits for a flow to complete has already let the attack through, and buffering whole flows at a network boundary is not viable performance-wise. Second, black-box realism: the defense may not assume any knowledge the attacker would not grant it, which means it must be trained and evaluated against realizable, protocol-compliant adversarial traffic rather than against attack-convenient synthetic perturbations. Third, realizable output: a flow the defense repairs must leave it as valid traffic, packets that parse and keep their protocol semantics, otherwise the defense has merely moved the realizability problem from the attacker to the defender.

Two boundaries complete the statement. DEPT complements the protected NIDS, it does not replace it: recognizing the underlying attacks themselves (a flood, a scan, a brute-force attempt) remains the downstream system's task, and DEPT answers only the narrower question of whether a flow has been tampered with on its way in. Since the defense sits inline, its decisions must also fit a real-time latency budget: the per-flow cost of detection, and of purification when it triggers, has to stay low. This budget shapes the design choices of Chapter 4; measuring the delivered system against it is left to future work (Section 6.4).

1.3 Work Objectives and Challenges

The problem statement was broken into four objectives, which structure both the design of Chapter 4 and the research questions that organize the evaluation of Chapter 5:

- **O1 (shared traffic representation).** Find a representation of raw traffic that stays close to the packets themselves, so that model outputs can be turned back into real traffic, and that architecturally different learners can consume at different temporal granularities, from individual packets to aggregated flow statistics.
- **O2 (heterogeneous detection).** Build a detection ensemble whose members belong to different architectural families, so that a perturbation shaped against one member does not automatically transfer to the others, and fuse their different opinions into a single per-packet verdict.
- **O3 (input purification).** Learn to reconstruct a flagged flow: remove the adversarial manipulation and return the flow to its clean appearance, while keeping the result a valid, sendable packet sequence.
- **O4 (streaming-aware evaluation).** Evaluate the system the way it would be used: on partial flows as they arrive, with metrics that expose how early a verdict becomes reliable, rather than end-of-flow accuracy alone.

The setting makes these objectives harder than they may look, and three difficulties deserve to be named early. The first is imbalance: public attack corpora are dominated by high-rate floods, so a handful of traffic sources contributes the overwhelming majority of flows, and both training and evaluation have to be designed so that these sources neither drown out the rest nor flatter the aggregate numbers. The second is the thinness of the signal: a manipulation confined to packet timing on a high-rate flood barely moves any observable feature, and no detector can find a footprint where almost none exists; Chapter 5 quantifies this limit openly instead of averaging it away. The third is that the defense inherits the attacker's constraint: purification edits traffic, and edited traffic must remain realizable, which rules out treating flows as feature vectors and directly forces the representation choices of O1. Cutting across all three runs the latency constraint: whatever the pipeline does, it must do at a streaming pace.

1.4 Rationale and Contributions

The design of DEPT rests on two decisions, whose full justification occupies Chapters 2 and 3 but whose logic can be given in a few lines. The first concerns detection. A black-box attacker's practical weapon is transferability: perturbations crafted against a substitute model tend to carry over to the target when

both expose similar, differentiable decision surfaces [11]. DEPT's detector therefore stacks two learners from different architectural families, a Random Forest (RF), whose decision function offers no gradient at all, and a Temporal Convolutional Network (TCN), which reads the raw packet sequence, fused at every packet by a small attention-based meta-learner [14]. What moves one member's verdict does not necessarily move the other's, and a single gradient signal cannot cross both.

The second decision concerns what to do with a flagged flow. Dropping it would turn every false positive into denial of service for a legitimate client, so DEPT purifies instead, in the input-transformation tradition started by Defense-GAN [15]. The purifier is a causal Transformer that predicts a residual correction over the packet sequence, rather than a Generative Adversarial Network (GAN) or a diffusion model: flows are sequences, which is what self-attention models well; causality preserves the streaming property, and a residual head makes "change nothing" the default behaviour.

Against this rationale, the thesis makes five contributions:

- A streaming packet-to-tensor pipeline that turns raw captures into a masked, bit-level representation of each flow, emitted at two temporal granularities, so that architecturally different learners share one view of the same traffic (Section 4.3).

- A heterogeneous detection ensemble, an RF and a TCN fused per packet by an attention meta-learner, together with a streaming comparison that scores every model by how early, and not only how well, it separates manipulated from clean flows (Sections 4.4 and 5.5).

- A causal purification Transformer coupled to an explicit realization step: flagged flows are corrected in feature space, written back into protocol-compliant packets, and validated with a standard protocol dissector (`tshark`). On the sources where the manipulation leaves a footprint the protected detector can see, purification recovers most of the detection the attack destroyed, confirmed not through DEPT's own detector but through the independent NIDS each attack was built to evade (Sections 4.5 and 5.6).

- An end-to-end deployment baseline that measures DEPT as what it is, a defense placed in front of a protected NIDS: the full pipeline is run over clean, TANTGen-perturbed, and DEPT-defended traffic, and each is scored through the real NIDS the attack targets, Kitsune for the Kitsune-native attacks and TANTGen's ML-NIDS for the CIC attacks, reporting the attack-success rate the defense removes and the detection it restores (Section 5.8).

- An adversarial evaluation corpus and its honest accounting: ten benign traffic sources drawn from three public datasets (Kitsune, CIC-IDS2017, and CIC-DDoS-2019 [16–18]), each manipulated by TANTGen in three perturbation modes, evaluated end to end with an explicit per-source analysis of where the defense works, where it is inert, and why (Section 4.3 and Chapter 5).

1.5 Organization of the Document

The remainder of this document is organized as follows. Chapter 2 presents the background: learning-based intrusion detection and traffic representation, adversarial machine learning and the realizability constraints that distinguish it in the network domain, and the ensemble concepts behind DEPT's detector. Chapter 3 surveys related work on the attack and the defense sides, treats TANTGen in the detail its role here requires, and states the gap DEPT fills. Chapter 4 presents DEPT itself: the threat model, the system overview, the corpus and packet-to-tensor pipeline, the detection ensemble, and the purification and realization stages. Chapter 5 evaluates the delivered system against two research questions, reporting the cases where it does poorly with the same weight as those where it excels. Chapter 6 closes with the answers to the research questions, the system's limitations, and future work.

2

Background

Contents

2.1 Machine Learning for NIDS 7
2.2 Adversarial Machine Learning 12
2.3 Ensemble Learning for Robustness 16
2.4 Summary 18

DEPT combines three main ingredients: a bit-level representation of network traffic, a heterogeneous detection ensemble, and a learned purifier. This chapter presents the concepts each of them builds upon. Instead of a broad survey of machine learning for security, the focus is kept on what the system actually relies on, so that the design decisions of Chapter 4 follow from the principles introduced here.

2.1 Machine Learning for NIDS

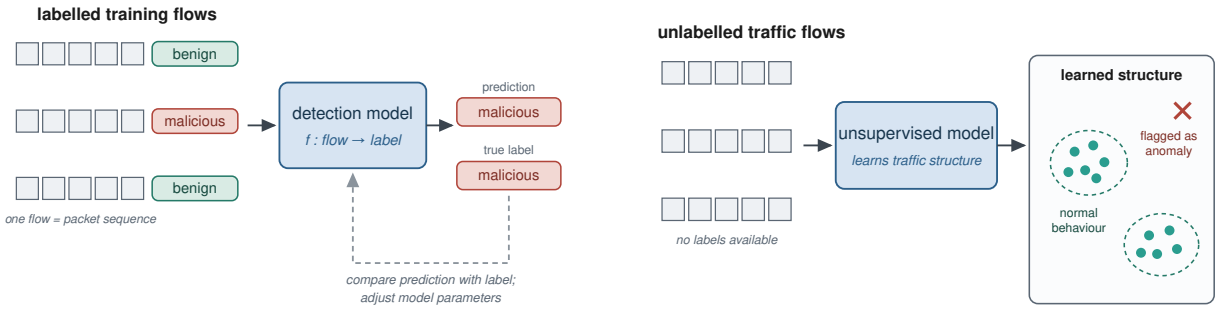
A NIDS inspects network traffic in order to separate benign activity from attacks. Classical systems match traffic against hand-written signatures of known threats, which makes them precise on catalogued attacks but unable to detect anything absent from the rule set, including minor variants of existing malware. ML reformulates detection as a data-driven problem: instead of enumerating signatures, a model

learns the statistical structure that separates benign from malicious traffic, and can therefore generalize to variants and to previously unseen attacks [1, 2]. This shift supports a large body of modern detection research [3, 4], and it defines the setting in which DEPT operates: every component of the system is a learned model.

2.1.1 Learning Paradigms

Learning-based intrusion detection is usually organized by the kind of supervision a model requires. Four paradigms recur in the literature. Every component of DEPT is trained under the first; the second matters here because the independent NIDS DEPT is evaluated in front of is an unsupervised one.

Supervised learning trains on labelled data, where each example is marked benign or malicious, and learns a mapping from input features to labels (Figure 2.1). Random forests, support vector machines, and deep neural networks all follow this paradigm [1, 19], and it is the main approach in DEPT: the two base detectors, the meta-learner that fuses them, and the purifier are all trained under supervision. Unsupervised learning uses no attack labels. It models the normal behaviour of a network and flags departures from that baseline as anomalies, which makes it attractive for catching zero-day attacks that a supervised model never saw during training. The canonical example in this domain is Kitsune, an ensemble of autoencoders that scores each packet by how poorly it reconstructs under a model of benign traffic [16]. Semi-supervised learning sits between the two, exploiting a small labelled set together with abundant unlabelled data, and reinforcement learning replaces the fixed dataset with an agent that learns by interacting with an environment. DEPT uses neither, although reinforcement learning is the natural framework for the continuous-adaptation extension discussed in Chapter 6.



(a) Supervised learning: labelled flows train a model that maps a flow to a label.

(b) Unsupervised learning: with no labels, the model learns the structure of the traffic and flags what departs from it.

Figure 2.1: The two supervision regimes most relevant to DEPT, on network flows. All of DEPT's components are supervised; the unsupervised idea of modelling normal behaviour and flagging departures from it is what the purification literature builds on, and what the independent NIDS of Chapter 5 does.

2.1.2 Feature Engineering and Representation

The performance of a learned detector depends directly on the representation it consumes. Feature engineering, the process of turning raw packets into numerical vectors, places an upper bound on what any model can learn [20, 21]. Before discussing representations, one term should be fixed: throughout this document, a flow is the sequence of packets that share the same five-tuple (source and destination Internet Protocol (IP) addresses, source and destination ports, and transport protocol), ordered by arrival time.

Two axes organize the design space of traffic representations. The first is granularity. Static, packet-level features are read from individual headers (addresses, ports, flags); they are cheap enough to extract at line rate, but they carry no temporal context. Dynamic, flow-level features aggregate a sequence of packets into statistics such as mean packet size, inter-arrival times, or flow duration. These capture behaviour, but at the cost of latency, since the system must wait for the flow to progress [12]. The second axis concerns the space in which a perturbation is expressed: the traffic-space of actual packets on the wire versus the feature-space of derived vectors. This distinction becomes central once adversaries are considered (Section 2.2.3). DEPT works at both granularities, and deliberately keeps its features close to the traffic-space so that its outputs can be turned back into real packets.

Rather than hand-designing features, DEPT represents traffic with **nPrint**, a bit-level encoding of packet headers [22]. Hand-crafted feature sets usually assume in advance which fields matter, and they are tied to particular protocols: a tool that computes flow statistics for Transmission Control Protocol (TCP) says nothing useful about an Internet Control Message Protocol (ICMP) flood. nPrint instead maps each bit of a packet's headers to one feature in a fixed-width vector, using a tri-state encoding: a value of 1 or 0 records a header bit that is set or unset, and -1 marks a field that belongs to a protocol layer absent from that packet. The representation is uniform across protocols, requires no manual feature design, and can be consumed directly by models that learn their own features. Bit-level encodings of this kind have proven effective for traffic analysis in settings where hand-crafted features generalize poorly [22–24]. DEPT encodes the IPv4, TCP, User Datagram Protocol (UDP), and ICMP headers, producing a 1088-bit vector ($480 + 480 + 64 + 64$), and appends two derived per-packet scalars, the z-scored Inter-Packet Arrival Time (IPAT) and a direction bit, to obtain a 1090-dimensional per-packet token. Figure 2.2 illustrates the encoding.

A bit-level header vector inevitably includes fields that identify the particular hosts and ports of a capture: the source and destination IP addresses, the transport ports, and the IP identification field. If these fields are left intact, a model can reach high benchmark accuracy simply by memorizing which addresses and ports appear in the attack traces, instead of learning the attack behaviour itself; this shortcut inflates test scores but does not survive deployment on a different network [12, 25]. DEPT therefore masks these identity fields, zeroing the 144 bits that encode them (Figure 2.2). Masking forces

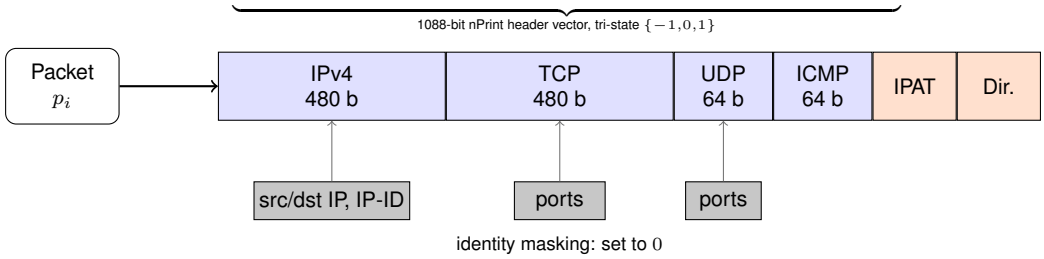


Figure 2.2: The per-packet representation used by DEPT: the four header blocks encoded bit-by-bit into 1088 tri-state cells following nPrint [22], the identity fields masked to zero, and the two derived scalars appended to give a 1090-dimensional token.

the detectors to rely on behavioural structure, such as packet sizing and timing, and it directly supports the claim that detection transfers across deployments instead of overfitting to one capture.

The same tokens feed two representations at different rates, a design choice matched to the two kinds of base learner that DEPT uses (Table 2.1). The TCN consumes the full per-packet token stream, one 1090-dimensional vector per packet. The random forest instead consumes prefix-checkpoint statistics: at selected packet positions, six base features (inter-arrival time, packet size, payload length, payload entropy, TCP flags, and direction) are summarized by their mean, standard deviation, minimum, and maximum over the flow prefix seen so far, giving a compact 24-dimensional vector. Checkpoints are emitted at packet 2 and then every tenth packet. The early checkpoint is deliberate: some attacks generate extremely short flows. A port scan, for example, averages roughly two packets per flow, typically a single SYN and its RST, so without the checkpoint at packet 2 the forest would never issue a verdict for precisely the flows that a scan consists of. Feeding one learner a dense per-packet sequence and the other a sparse, aggregated summary is also a source of the ensemble diversity that Section 2.3 argues to be valuable.

Table 2.1: The two representations DEPT derives from the same masked nPrint tokens.

Representation	Consumer	Emission rate	Dimension
Per-packet token	TCN	every packet	1090 (nPrint $+$ IPAT $+$ direction)
Prefix checkpoint	Random Forest	packet 2, then every 10	24 (mean/std/min/max of 6 features)

2.1.3 Model Architectures Relevant to DEPT

DEPT combines three architecture families, chosen for complementary properties rather than for raw accuracy alone.

A – Random forests. An RF is an ensemble of decision trees trained by bagging (Section 2.3.1): each tree recursively partitions the feature space with axis-aligned threshold tests, and the forest aver-

ages their votes [26, 27]. Forests are a strong, low-cost fit for tabular and statistical features and need no feature scaling [28], which is why DEPT applies one to the 24-dimensional checkpoint vectors. Their decisive property for this work, however, is structural: a forest's decision function is constant over axis-aligned regions and therefore non-differentiable. It exposes no informative gradient that an attacker can follow, a property whose relevance for adversarial robustness is developed in Section 2.3.2.

B – One-dimensional and temporal convolutional networks. A Convolutional Neural Network (CNN) slides learnable kernels over its input to detect local patterns; in one dimension it processes sequences, extracting recurring motifs from a packet stream such as header patterns or sizing signatures [23, 24] (Figure 2.3). A TCN specializes this design for long sequences through two devices [29]. Dilated convolutions insert a growing stride between kernel taps, so the receptive field expands with depth without a corresponding blow-up in parameters [30]. Causal convolutions pad the input so that the output at position t depends only on positions $\leq t$, never on future packets. Causality is what makes the model valid for streaming inference: it can emit a verdict after each packet arrives. DEPT's TCN stacks three residual blocks of 128 channels with kernel size $k = 3$ at dilations 1, 2, and 4, with two convolutions per block. Its receptive field is thus

$$\mathrm{RF} \;=\; 1 + 2\,(k-1)\sum_{i=0}^{2} 2^{i} \;=\; 1 + 2 \cdot 2 \cdot (1+2+4) \;=\; 29 \text{ packets}, \tag{2.1}$$

so a verdict at any packet integrates evidence from that packet and the 28 preceding it (Figure 2.4).

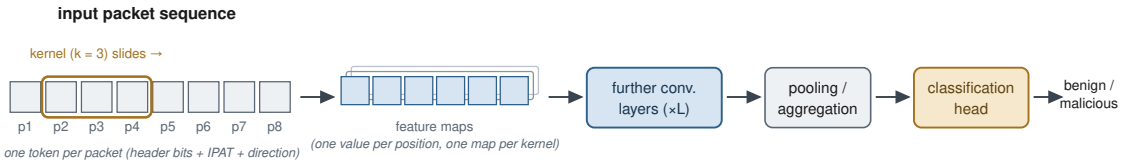


Figure 2.3: A one-dimensional convolutional pipeline over a packet sequence: kernels slide along the sequence and produce per-position feature maps, which further layers, pooling, and a classification head turn into a verdict. DEPT's temporal convolutional network specializes this design with dilated, causal convolutions.

C – Transformers. The self-attention mechanism lets every position in a sequence attend to every other position, weighting them by a learned measure of relevance [31]. Each input position is projected into a query, a key, and a value vector; collecting these as the rows of matrices Q, K, and V, attention computes

$$\mathrm{Attention}(Q, K, V) \;=\; \mathrm{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V, \tag{2.2}$$

where d_k is the key dimension, so the output at each position is a weighted combination of the values of the positions it attends to. This captures long-range dependencies that a fixed convolution window

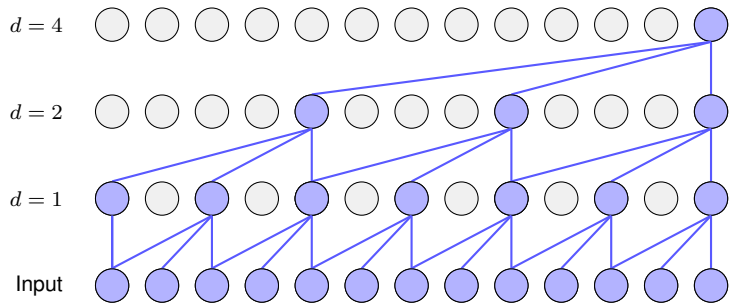


Figure 2.4: Dilated causal convolutions. Highlighted nodes trace the dependency cone of the rightmost output: dilation grows with depth $(1, 2, 4)$, so the receptive field widens while every output still depends only on the present and past. Schematic, with one convolution per block drawn; DEPT stacks two, giving the 29 packets of Equation (2.1).

cannot. Two arrangements are relevant here. Encoder-only models attend bidirectionally over the entire sequence, which suits classification and representation tasks, including learned traffic analysis [32, 33]. Decoder-only models apply a causal mask so that each position attends only to the past (Figure 2.5); this is the arrangement behind autoregressive generation, but the mask is what any task whose output at a position may depend only on that position's past requires, generative or not. Causality also enables key–value caching: the keys and values of past positions in Equation (2.2) do not change as new tokens arrive, so they can be stored and reused instead of reprocessing the whole prefix on every packet. Both of DEPT's Transformer components are decoder-only stacks, since both must act on a packet with only its predecessors in hand: the meta-learner fuses the base detectors' per-packet outputs (Section 2.3.1) and the purifier rewrites a perturbed packet sequence (Chapter 4). Neither is autoregressive, as neither feeds its own output back as input; causality buys them streaming validity rather than generation.

2.2 Adversarial Machine Learning

The same flexibility that lets a learned model generalize also exposes it to manipulation. An adversarial example is an input perturbed by a small, deliberate amount so that the model misclassifies it, often while remaining nearly indistinguishable from a clean input [5, 6]. For a NIDS, adversarial examples are the mechanism of evasion: an attacker reshapes malicious traffic just enough for it to be labelled benign [7, 34]. Adversarial Machine Learning (AML) studies both how such manipulations are constructed and how models can be hardened against them.

2.2.1 Threat Taxonomy

Attacks are classified along two axes: what the adversary knows, and what the adversary wants.

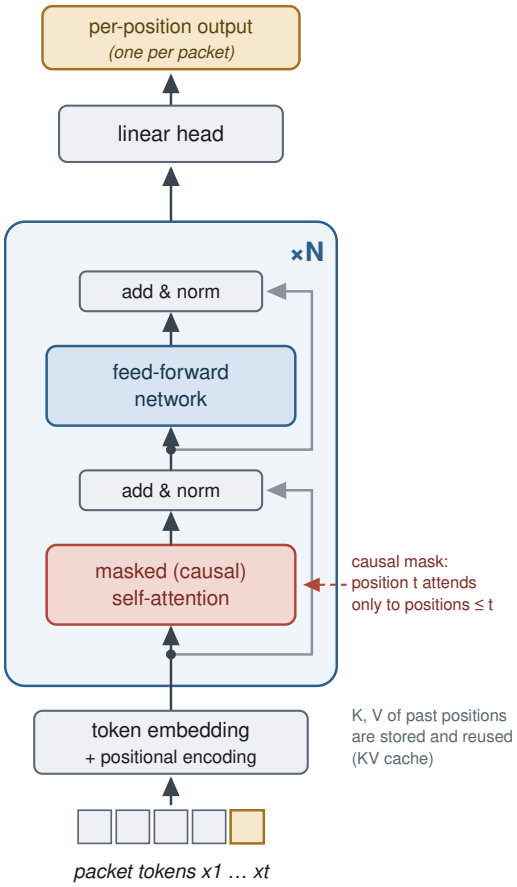


Figure 2.5: A decoder-only Transformer block over a stream of packet tokens: masked self-attention, then a feed-forward sublayer, with residual connections and layer normalization, stacked N times. The causal mask restricts each position to its past, which is what makes the design valid for streaming inference. Both of DEPT's Transformer components use this block.

Along the knowledge axis, a white-box attacker has full access to the model, including its architecture and weights, and can compute gradients of the loss with respect to the input to craft precise perturbations. A gray-box attacker knows only part of the pipeline, such as the feature set. A black-box attacker can only query the system and observe its outputs. The black-box setting is the realistic one for a deployed NIDS, which an attacker probes as an oracle, and such attacks typically rely on transferability: an adversarial example crafted against a substitute model often fools the target as well [11]. Along the goal axis, poisoning corrupts the model during training, whereas evasion manipulates inputs at inference time against an already-trained model. DEPT is designed for the black-box evasion threat, which is the case that dominates real deployments (Chapter 4).

Network intrusion detection is harder to attack, and to defend, than the image classification setting where adversarial examples were first studied, because of a constraint with no peer in the pixel domain. Any pixel of an image can be modified slightly and the result is still a valid image. A network feature vector cannot be modified freely: a perturbation must correspond to a packet or flow that is actually valid on the wire. Recovering a realizable packet from a desired feature vector is the **IFM** problem, and it is frequently constrained or computationally intractable [9,10,35]. The IFM problem is the defining difficulty of network adversarial machine learning, and it shapes attacks and defenses alike. In particular, the gradient-based methods that dominate the image setting, such as the Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD) [6, 36], produce feature perturbations that don't need to correspond to any sendable packet, which limits their direct applicability in this domain.

2.2.2 Defense Paradigms

Defenses against evasion fall into a few broad families, with different strengths for traffic data. Adversarial training augments the training set with adversarial examples so that the model learns to classify them correctly [36]. It is effective against the perturbations it is trained on, but it is costly, it can depress accuracy on clean traffic, and in networking it is only meaningful if the adversarial examples are themselves realizable rather than impossible feature vectors. DEPT follows this line in spirit, since its detectors are trained on realizable adversarial traffic rather than on synthetic feature perturbations (Chapter 4); the fundamental difference is that DEPT's goal is to distinguish naturally occurring traffic from adversarially perturbed traffic, not to classify both as malicious.

Input transformation, or purification, is the family DEPT builds its purifier on. Instead of hardening the classifier, it transforms each input before classification in order to remove the perturbation, projecting the sample back onto the manifold of normal traffic, as Figure 2.6 illustrates. The idea originates with Defense-GAN, which projects each input onto the manifold captured by a generative adversarial network [15], and it has since been realized in the network domain with diffusion models [37, 38] and reconstruction-based Transformer autoencoders [33]. DEPT's purifier pursues the same goal without

being a generative model itself: it is a causal Transformer denoiser, trained on matched adversarial-to-clean pairs rather than on a model of the benign distribution it could sample from (Chapter 4).

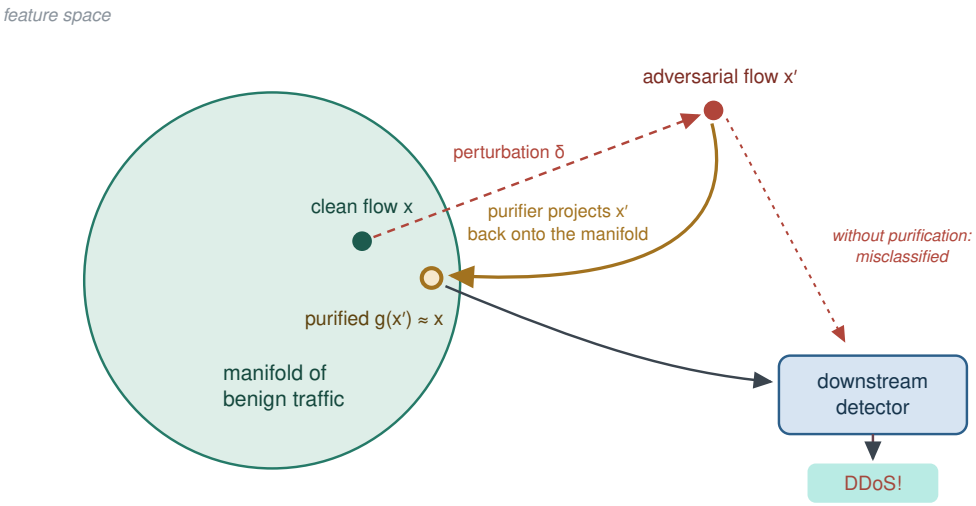


Figure 2.6: Purification as projection onto the benign manifold. A perturbation δ moves a clean flow x off the manifold, and the detector misclassifies the result x'. The purifier g projects x' back, so the detector receives an input close to x. The idea originates with Defense-GAN [15].

DEPT adopts neither of the two remaining families. Defensive distillation smooths the decision surface so that useful gradients become harder to find [39], but it has been largely circumvented and says nothing about realizability. Certified robustness guarantees that no perturbation within a bounded region changes a prediction, through randomized smoothing [40, 41] or Lipschitz and edit-distance bounds [42]; such guarantees are hard to scale to the high-dimensional, discrete structure of traffic, and they secure a formal radius rather than the traffic-space validity that is central to this work.

2.2.3 Realizability and Protocol Constraints

The gap between a valid feature vector and a valid packet is a central difficulty of network adversarial machine learning. A perturbation can move a feature vector anywhere in $\mathbb{R}^n$, but a packet must satisfy hard structural constraints: checksums must match the bytes they cover, header-length fields must be equal to the actual lengths, control flags must be mutually consistent, and sequence numbers must be ordered. A perturbation that ignores these constraints is unrealizable: it describes no packet that can be sent, so evaluating a detector against it overestimates the real threat [9, 10]. Recent attack research enforces realizability explicitly, whether by verifying every perturbation against protocol constraints with an SMT solver, as in PANTS [43], or by generating traffic that is protocol-compliant by construction, as in GPMT [35] and TANTGen [13].

The constraint applies to both sides, which is what makes it a design concern for DEPT rather than a property of attacks alone. A defense that edits traffic, as a purifier does, must itself re-emit valid

packets; otherwise its output is unusable, or is trivially detected as malformed. The step from feature-space back to traffic-space, which we call realization, is therefore treated as a first-class part of the design. The nPrint representation is an asset here: because it is close to a bijective, bit-level image of the packet headers, a purified nPrint vector can be decoded back into a concrete packet and checked for on-the-wire validity, which DEPT does with packet-level tools (Chapter 4).

2.3 Ensemble Learning for Robustness

An ensemble combines several base models into a single predictor, on the premise that a collective decision is more accurate and more robust than that of any single learner [28, 44]. Beyond accuracy, ensembles are of particular interest for security, because diversity among the base models can be turned directly against adversarial transferability.

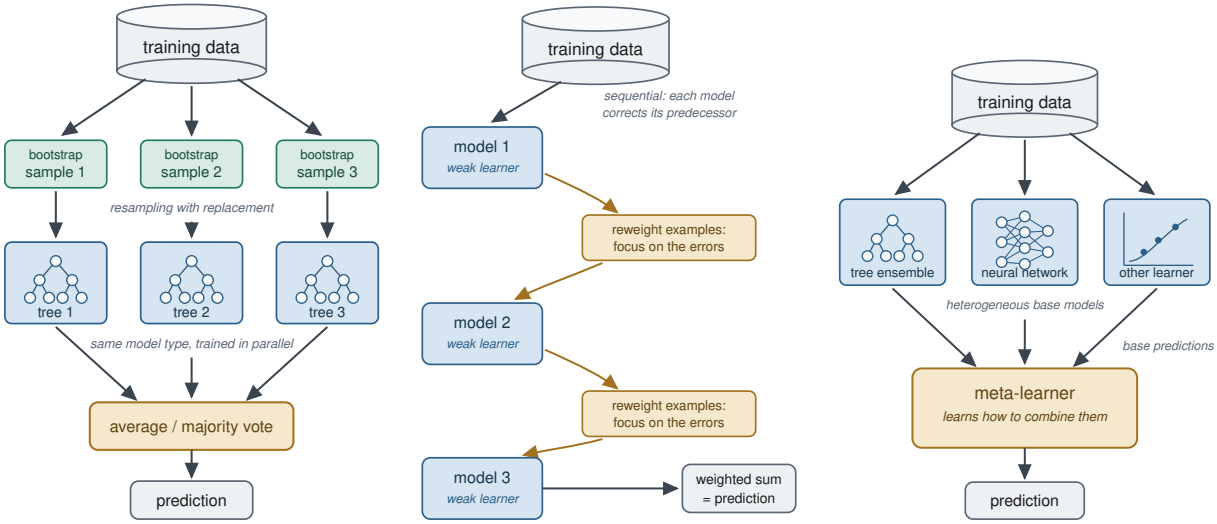
2.3.1 Bagging, Boosting, Stacking

Three combination strategies dominate (Figure 2.7). Bagging trains homogeneous models in parallel on bootstrap resamples of the data and averages their predictions, which reduces variance; the random forest is bagging applied to decision trees [26, 27]. Boosting trains models sequentially, each one focusing on the examples its predecessors got wrong, which reduces bias [45]. Stacking trains heterogeneous base learners and then trains a separate meta-learner to combine their outputs [14]. Stacking is most effective precisely when the base learners are diverse and make different kinds of errors, so that their combination covers more of the input space than any of them alone.

DEPT stacks a random forest and a temporal convolutional network, which is precisely the setting stacking was designed for. The two learners are heterogeneous in architecture and asynchronous in operation: the network emits a dense verdict on every packet, while the forest emits a sparse verdict only at its checkpoints, starting at packet 2. A fixed combination rule, such as averaging or majority voting, cannot express when each learner should be trusted. A learned meta-learner can: by assigning input and position-dependent weights to each base output, it acts as a dynamic gating mechanism rather than a static average [46, 47]. DEPT implements this meta-learner as a small attention model over the fused per-packet outputs, a choice that is further motivated by the transferability argument below and detailed in Chapter 4.

2.3.2 Gradient Masking and Transferability

Transferability is what makes black-box evasion practical: an attacker who cannot see the target model builds a differentiable substitute, crafts a perturbation against it with a gradient method such as PGD



(a) Bagging: parallel homogeneous models on resampled data.

(b) Boosting: sequential models, each correcting the last.

(c) Stacking: heterogeneous models fused by a meta-learner.

Figure 2.7: The three classical ensemble strategies. DEPT's detector is a stacking ensemble, chosen because its base learners are architecturally heterogeneous and emit predictions at different rates.

or the FGSM, and relies on that perturbation carrying over to the target [11]. Heterogeneous stacking acts directly on this mechanism. A differentiable member, such as the TCN, exposes a gradient that an attacker can follow. An RF does not: its decision function is constant over axis-aligned regions, so there is no gradient direction to exploit. Combining a differentiable and a non-differentiable model yields what the literature calls a rugged decision boundary, in which a perturbation optimized against the differentiable member does not necessarily move the forest's discrete verdict, and therefore transfers poorly across the ensemble, raising the attacker's cost [47–50].

This argument must be stated carefully, because the absence of a usable gradient is a form of gradient masking, and gradient masking is not robustness in the general case. A determined white-box adversary can estimate gradients numerically or resort to gradient-free search, and masked gradients have repeatedly produced a false sense of security when mistaken for genuine robustness [51]. The argument holds for DEPT precisely because its threat model is black-box: the value of a non-differentiable member is not an absolute barrier, but a measurable reduction in the transferability from whatever substitute the attacker is able to build without direct gradient access. Robustness is therefore treated as a property to be evaluated empirically, not as a guarantee to be assumed, a distinction that Chapter 5 takes up thoroughly.

2.4 Summary

The concepts in this chapter compose into the three pillars of DEPT. Representation: a bit-level nPrint encoding, stripped of identity fields and emitted at two rates, gives heterogeneous learners a shared, behaviour-focused, and realizable view of traffic (Section 2.1.2). Ensembling: heterogeneous stacking, fused by an attention meta-learner, combines a differentiable network with a non-differentiable forest to resist transferable gradient attacks under a black-box threat model (Section 2.3). Purification: an input-transformation defense, built as a causal Transformer, projects flagged traffic back onto the benign manifold while respecting the protocol constraints that keep its output realizable (Sections 2.2.2 and 2.2.3).

3

Related Work

Contents

3.1 Adversarial Attacks against ML-based NIDS 20

3.2 Training-time Defenses 22

3.3 Inference-time Defenses 24

3.4 Positioning of DEPT and Gap Analysis 25

The tension between learning-based detection and adversarial evasion has produced a large body of research on both sides. Chapter 2 introduced the concepts this work builds on; this chapter surveys the concrete systems, and reads each of them against the questions DEPT has to answer: what realistic attacks against a NIDS look like (Section 3.1), what can be done before deployment to harden a model (Section 3.2), and what can be done at inference time on the inputs themselves (Section 3.3). The survey is deliberately selective rather than exhaustive: most works are included when they shape a design decision of DEPT, or when some part of DEPT departs from them. Section 3.4 closes the chapter by placing the surveyed defenses and DEPT on a common map and stating the gap this thesis addresses.

3.1 Adversarial Attacks against ML-based NIDS

A defense is only meaningful relative to the attacks it expects, so the survey starts on the offensive side. Following the distinction of Section 2.1.2, attacks are divided by the space in which they operate: the feature-space of derived vectors, or the traffic-space of actual packets.

A – Feature-space attacks. The first wave of attacks against learned traffic detectors used the gradient-based methods of the image domain, FGSM and PGD among them [6, 36], onto NIDS feature vectors. Hashemi et al. [10] were among the first to evaluate state-of-the-art network detectors under such manipulations, and Tiki-Taka [8] made the threat concrete at scale: attacking deep learning-based intrusion detectors with a battery of black-box methods, it reached evasion rates of up to 35.7% against models that looked nearly perfect on clean benchmarks. Two lessons carry over to this work. First, a detector built from a single differentiable architecture inherits a smooth loss surface that gradient-guided search can exploit, directly or through a substitute model [11]; notably, the defenses that Tiki-Taka itself proposed, model voting and ensemble adversarial training, already point towards the ensemble route DEPT takes. Second, most of these attacks perturb feature vectors without asking whether any real packet sequence maps to the perturbed vector, which is exactly the realizability critique of Section 2.2.3 [9, 12]: their reported evasion rates are upper bounds obtained in a space an external attacker does not control.

B – Traffic-space attacks. The recent attack literature has therefore moved into the traffic-space, enforcing realizability either by verification or by construction. **GPMT** [35] generates practical malicious traffic with a GAN under little prior knowledge of the target, constraining its perturbations so that the traffic keeps its malicious function. **PANTS** [43] couples adversarial search with a Satisfiability Modulo Theories (SMT) solver, so that every candidate perturbation is formally checked against protocol and semantic constraints before it is accepted; the cost of solving is the price of the guarantee. **DeepRed** [52] embeds a GAN-based generator in a multi-stage command-and-control framework and shows that realizable, protocol-abiding attacks degrade the detection rate of ML-based NIDS by up to 20%, which settles that the threat survives the realizability constraint. Most recently, **NetDiffuser** [53] brought diffusion models to the attack side, injecting semantically consistent perturbations into flow features, although still under a white-box assumption. The trend across these works is clear: the attack side has converged on generative sequence models operating under explicit realizability constraints, and it is precisely to this class that the attacker of this thesis belongs.

3.1.1 TANTGen, the Attacker of This Thesis

TANTGen [13] is a Transformer-based adversarial network traffic generator developed in a sibling thesis, and it deserves a closer treatment than the works above because it is not merely related work: it is the concrete adversary that DEPT is trained against and evaluated on. Where GPMT and DeepRed rely on GANs and PANTS on solver-checked perturbations, TANTGen uses self-attention to model the sequential dependencies between the packets of a flow, and generates its evasions in the traffic-space directly, so that protocol compliance holds by construction rather than by after-the-fact verification. It operates in the black-box setting: it needs no access to the target detector's architecture, weights, or gradients, only to the detection outcomes it can observe while probing, and its evasion capability was demonstrated against the detectors its generation was calibrated on: Kitsune [16], the anomaly-based detector, for its Kitsune-native attacks, and its own suite of ML-NIDS flow classifiers for the CIC attacks. The manipulation itself is confined to two physical quantities of live attack traffic, packet timing and packet size, exposed as the three perturbation modes (`iat`, `size`, `both`) that structure the corpus of Section 4.3.

Taking TANTGen as the adversary has three consequences that run through DEPT's design. First, since the adversarial traffic is protocol-compliant by construction, a defense gains nothing from checking protocol validity; whatever signal exists must be found in the traffic's behaviour, its timing and sizing structure, which is what DEPT's learned detectors consume. Second, because the perturbation touches only timing and size, the detectable footprint can be thin, and in the timing-only mode genuinely faint; Chapter 5 quantifies this honestly. Third, the black-box setting makes transferability the operative attack channel, which is the scenario the heterogeneity argument of Section 2.3.2 is aimed at. The threat model of Section 4.1 formalizes these assumptions.

3.1.2 Gradient-guided Evasion, the Second Attacker

TANTGen is the adversary DEPT is trained on, but it cannot be the only one it is evaluated against, and the reason is thoroughness. DEPT's central design claim is that fusing a non-differentiable Random Forest with a differentiable TCN raises the cost of evasion because an attacker's gradient reaches only one of the two. With TANTGen we can demonstrate that the ensemble detects its perturbations, but it cannot measure the mechanism this specific claim rests on. Section 5.7 closes that gap with a complementary adversary drawn from the standard adversarial-ML toolkit.

The canonical instrument is PGD [36], which takes repeated gradient steps against a model's loss and projects each iterate back onto a feasible set. Its strength as an evaluation tool is that the projection step states the attacker's budget outright: the set of admissible perturbations is written down as an operator applied at every iteration, instead of being whatever the optimizer happens to produce once its hyperparameters are fixed. That explicitness matters because the opposite failure is easy to fall into. An

attack whose step size or iteration count is badly chosen will fail to find perturbations that do exist, and the defense it was pointed at will look more robust than it is; in the reported numbers, a weak attack and a strong defense are indistinguishable. Robustness evaluations therefore cross-check a hand-configured attack against an optimizer with no such knobs to misconfigure: AutoPGD [54] adapts its own step size as the attack proceeds, and both it and PGD are available in the Adversarial Robustness Toolbox (ART) library [55]. The literature on this failure mode is unambiguous: many published defenses have been shown to owe their apparent robustness to weak or badly-adapted attacks rather than to any real property [56, 57], which is why the evaluation of Section 5.7 reports its attack-strength controls alongside its results rather than in a footnote.

Two features of the network-traffic setting keep this from being a routine application of those tools. The perturbation is not a free vector in the model's input space: it must remain a sendable packet, so the attacker's variables are bytes of padding and seconds of delay, and the budget must be denominated in those units rather than in an ℓ_p ball over a tensor whose channels carry incommensurable scales. And the ensemble's two halves do not even read the same features, so a perturbation crafted against one must be replayed through the other's featurisation to be scored at all. Section 5.7 takes both constraints seriously, and Section 5.9.1 discusses what they imply for how the result should be read.

3.2 Training-time Defenses

Training-time defenses harden a model before it is ever deployed, by sanitizing its training data or by shaping its decision surface. Four lines of work are relevant here, in rather different degrees.

A – Data sanitization and poisoning defenses. The earliest defensive line protects the integrity of the training set itself. **Antidote** [58] replaces the principal-component subspace of an anomaly detector with robust statistics, so that poisoned flows that deviate from the benign subspace are rejected instead of taken in; its acknowledged difficulty is telling adversarial poisoning apart from the benign drift that network traffic produces naturally. A complementary result concerns bagging: Biggio et al. [49] showed empirically, and Chen et al. [50] later certified, that training each ensemble member on a random subset of the data dilutes the influence of any individual poisoned sample across the collective decision. DEPT excludes poisoning from its threat model (Section 4.1), so these defenses are not competitors; they matter here because they establish, from the training side, the same principle DEPT exploits at inference time, that ensemble diversity is a robustness mechanism in it of itself and not only an accuracy one.

B – Federated and backdoor defenses. As NIDS deployments move towards Federated Learning (FL) for privacy, the defensive focus shifts from the data to the model updates that clients contribute.

FedTGE [59] detects backdoors in federated graph learning through the topological energy of client models, **AlignIns** [60] inspects the directional alignment of local updates, and **CrowdGuard** [61] has clients validate candidate models against their private data inside trusted execution environments. When a model is found already compromised, unlearning techniques such as **SEAM** [62], **BayBFed** [63], and **PBP** [64] scrub the backdoor from the weights at a fraction of the cost of retraining. All of these defend the training pipeline against an insider or a poisoning adversary; they are surveyed for completeness and treated as complementary rather than as alternatives.

C – Ensemble-based hardening. A third line builds robustness into the model by combining several learners, which is the training-time ancestor of DEPT's detection stage. Beyond Tiki-Taka's voting ensembles [8], **TRS** [48] showed that low transferability among ensemble members can be promoted directly, by regularizing for gradient diversity and model smoothness during training, and **Stewart** [47] demonstrated, in the domain of predictive maintenance, that a stacking ensemble with a learned meta-model withstands white-box attacks substantially better than its individual members. **Mateen** [65] takes the ensemble idea in the direction of distribution shift, maintaining an adaptive committee of autoencoders for network anomaly detection, and broader benchmarking frameworks confirm the accuracy benefits of ensembling for intrusion detection in general [44]. What nearly all of these have in common is that their members are drawn from the same differentiable family, so a gradient exists for every member and the ensemble's protection rests on decorrelating those gradients. Heterogeneity across families, mixing a non-differentiable forest with a differentiable network so that part of the ensemble offers no gradient at all, is far less explored, and it is the specific form of diversity DEPT's detector is built on (Section 4.4).

D – Realizable adversarial training. Adversarial training [36] hardens a model by adding adversarial examples into its training set, but in the network domain it is only as good as the examples themselves: a model trained against impossible feature perturbations is hardened against a threat that cannot occur, the IFM problem of Section 2.2.1 seen from the defender's side. The works of Section 3.1 close this gap by supplying realizable adversarial data, and PANTS in particular demonstrates that inserting its solver-verified examples into the training loop yields robustness that persists against attacks not seen during training [43], while DeepRed argues the same necessity from the red-team side [52]. DEPT's corpus construction follows this line: its detectors are trained on realizable, black-box adversarial traffic generated by TANTGen, not on synthetic feature noise (Section 4.3). The goal, however, differs. Adversarially trained classifiers aim to keep classifying perturbed attacks as attacks; DEPT aims to recognize the manipulation itself, and then to undo it, which places its second stage in the inference-time family surveyed next.

3.3 Inference-time Defenses

Inference-time defenses act at deployment, on the inputs a model receives. Three groups are relevant: certifying predictions against bounded perturbations, purifying inputs before classification, and detecting adversarial inputs by auxiliary signals.

A – Certified robustness. Certification promises that no perturbation within a bounded region can flip a prediction. **CertTA** [41] made randomized smoothing practical for traffic analysis by drawing its smoothing noise jointly over the modalities an attacker actually perturbs, packet timing, sizing, and insertion, and derives robustness regions that are meaningful for traffic models. **LipsLev** [42] obtains deterministic certificates under a bounded Levenshtein distance in a single forward pass, several orders of magnitude faster than smoothing-based verification. The guarantee is valuable but bounded by construction: a certificate covers a radius, and a TANTGen-style manipulation is not constrained to stay inside one, since a flow's timing can be stretched by legitimately large amounts while remaining perfectly realizable.

B – Generative purification. The purification family goes back to **Defense-GAN** [15], which projects each input onto the manifold captured by a generative model before classification. Its modern descendants are mostly diffusion-based, and mostly proven in the image and text domains: **ADBM** [38] constructs a direct reverse bridge from the adversarial to the clean distribution, **CausalDiff** [66] disassociates attack-specific factors from label-causal ones during generation (its notion of causality concerns factor attribution, not the temporal masking DEPT uses), and **DiffuseDef** [67] iteratively denoises the hidden states of a text classifier. In the network domain the idea has been instantiated more recently: **AdvPurRec** [37] reconstructs traffic features with a diffusion model before they reach the NIDS, Merzouk et al. [68] systematically maps the trade-offs that governs such purifiers, more diffusion noise removes more of the perturbation but degrades clean accuracy, and **GTAE-IDS** [33] applies a graph-Transformer autoencoder to reconstruct traffic representations for real-time detection.

DEPT's reconstructor (Section 4.5) shares this family's goal but departs from it on three axes, each a deliberate choice rather than an oversight. The first is architectural: diffusion purifiers pay an iterative denoising chain, tens of forward passes, for every input, which sits poorly with a per-packet latency budget, whereas DEPT uses a causal Transformer that produces its correction in a single pass and defaults to the identity (no-op) through its residual head. The trade is explicit: DEPT gives up the broad distribution coverage of a diffusion process in exchange for a model that fits streaming deployment. The second axis is supervision: purifiers in the Defense-GAN tradition learn from clean data alone and purify by projection, while DEPT exploits the matched adversarial-to-clean flow pairs its corpus provides, a stronger form of supervision that is unavailable in general but natural in a paired-generator setting supplied by

TANTGen. The third axis is the one this thesis considers decisive: every network purifier surveyed above stops at the feature vector, and none of them re-emits traffic. A purified vector that corresponds to no sendable packet sequence has simply moved the realizability problem from the attacker to the defender. DEPT treats the step back into the traffic-space, realization followed by dissector-level validation, as part of the defense itself (Section 4.5.3).

C – Constraint-based and auxiliary detection. A final group detects adversarial inputs without generative machinery. **NIDS-CBAD** [69] flags violations of inter-feature domain constraints, for instance a payload size inconsistent with a packet count, on the observation that naive feature-space attacks routinely break such invariants. **AMDS** [70] fuses ensemble disagreement, predictive uncertainty, and distributional anomaly scores into an adversarial-input detector with attack-adaptive weights. DEPT adopts neither as its detection principle, for a reason that follows directly from Section 3.1.1: its adversary emits protocol-compliant traffic by construction, so constraint checks are blind to it, and the constraint family defends against exactly the naive attacks that the realizability literature has already moved past. The disagreement signal of AMDS is closer in spirit to this work; DEPT's meta-learner can in fact be read as a learned, per-packet version of it, fusing the opinions of two deliberately different base learners (Section 4.4.3).

3.4 Positioning of DEPT and Gap Analysis

Figure 3.1 arranges the surveyed defenses along the two axes that organized this chapter, the phase at which a defense acts and the space in which it operates, and Table 3.1 records the same works with their strategies and techniques. Read together, they make the pattern of the literature visible. The training-time corner is well populated, but almost entirely in the feature-space; PANTS is the exception that carries realizability into training. The inference-time work concentrates on purification, yet every network purifier acts on feature vectors and returns feature vectors. Traffic-space thinking at inference time exists, in CertTA's certified regions, but certification bounds a radius rather than undoing a manipulation.

The gap this thesis addresses is the combination. Of the surveyed works, none brings together (a) training data that is realizable and generated under a black-box threat model, (b) an ensemble that is heterogeneous across architectural families, pairing a non-differentiable with a differentiable learner to resist transfer, and (c) a purification stage whose output is realized back into protocol-compliant packets. Each property exists in isolation: PANTS holds (a) but hardens classifiers and performs no purification, TRS and Stewart hold (b) within homogeneous differentiable families or outside the network domain, and AdvPurRec and GTAE-IDS hold (c) up to the feature vector, on single datasets and without a streaming constraint. DEPT composes all three and is evaluated end to end, against a live generative adversary

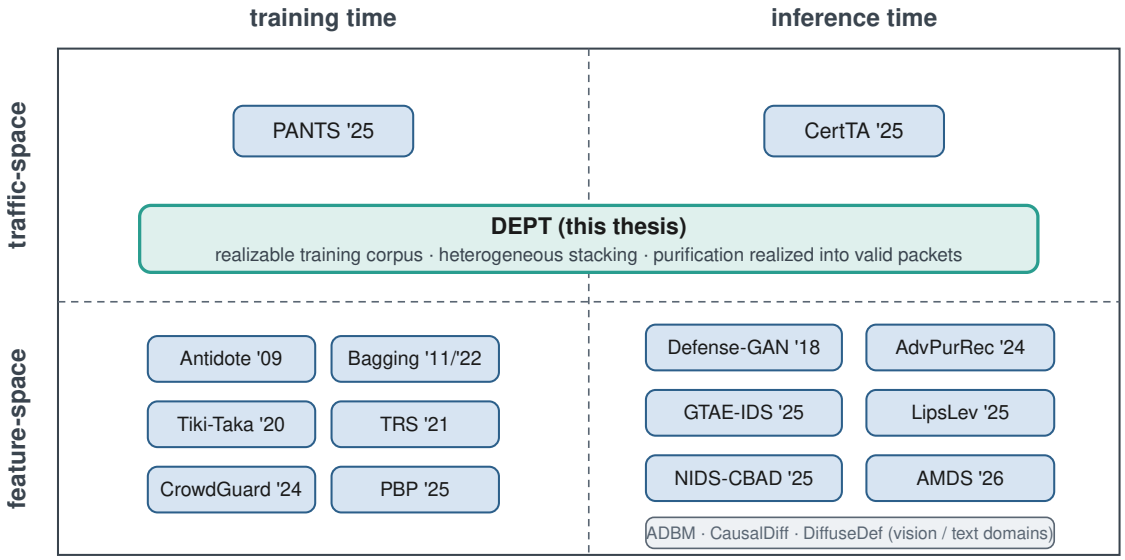


Figure 3.1: The surveyed defenses arranged by the phase at which they act (training time against inference time) and the space they operate in (feature-space against traffic-space). Prior defenses cluster in the feature-space row; PANTS brings realizability to training, and CertTA to certification. DEPT spans both phases in the traffic-space row: it is trained on realizable adversarial traffic and purifies flagged flows back into valid packets at inference time.

Table 3.1: Comparison of the surveyed adversarial defenses and of DEPT. Phase: $\mathcal{T}$ training-time, $\mathcal{I}$ inference-time. Scope: Φ feature-space (abstract vectors), Ψ traffic-space (realizable packets).

System	Year	Strategy	Phase	Scope	Techniques
Antidote [58]	2009	Statistical sanitization	$\mathcal{T}$	Φ	Robust statistics
Bagging defense [49, 50]	2011	Poisoning defense	$\mathcal{T}$	Φ	Bagged ensemble
Tiki-Taka (defense) [8]	2020	Ensemble hardening	$\mathcal{T}$	Φ	Voting, adv. training
TRS [48]	2021	Ensemble hardening	$\mathcal{T}$	Φ	Gradient diversity
CrowdGuard [61]	2024	FL backdoor defense	$\mathcal{T}$	Φ	TEE, clustering
PBP [64]	2025	Backdoor unlearning	$\mathcal{T}$	Φ	Neuron masking
PANTS [43]	2025	Adversarial training	$\mathcal{T}$	Ψ	SMT-verified examples
Defense-GAN [15]	2018	Purification	$\mathcal{I}$	Φ	GAN projection
AdvPurRec [37]	2024	Purification	$\mathcal{I}$	Φ	Diffusion
GTAE-IDS [33]	2025	Reconstruction detection	$\mathcal{I}$	Φ	Transformer autoencoder
CertTA [41]	2025	Certified robustness	$\mathcal{I}$	Ψ	Multi-modal smoothing
LipsLev [42]	2025	Certified robustness	$\mathcal{I}$	Φ	Levenshtein certificates
NIDS-CBAD [69]	2025	Adversarial detection	$\mathcal{I}$	Φ	Domain constraints
AMDS [70]	2026	Adversarial detection	$\mathcal{I}$	Φ	Ensemble signals
DEPT (this thesis)	2026	Detection + purification	$\mathcal{T}+\mathcal{I}$	Ψ	Realizable corpus, heterogeneous stacking, causal Transformer, traffic-space realization

rather than replayed perturbations, on a corpus built over ten benign traffic sources and three perturbation modes, with metrics that respect the streaming setting (Chapter 5). One further line of the literature is acknowledged but not pursued: reinforcement learning is increasingly used to make security responses adaptive [71–73], and a defense that retrains against an evolving generator is the natural continuation of this work; DEPT as delivered is a static defense, and Chapter 6 frames that continuation as future work. The architecture that occupies this gap is the subject of Chapter 4.

4

Solution: DEPT Architecture

Contents

4.1 Threat Model 30
4.2 System Overview 31
4.3 Dataset Construction and Packet-to-Tensor Pipeline 33
4.4 Primary Detection: Heterogeneous Stacking Ensemble 35
4.5 Input Purification: Reconstructor 37
4.6 Implementation and Tools 40

This chapter presents DEPT (Defense via Ensemble and Purification Transformer), the system this thesis proposes and builds. DEPT composes the three pillars introduced in Chapter 2 into a single streaming defense: a bit-level traffic representation emitted at two rates, a heterogeneous detection ensemble fused by an attention meta-learner, and a causal purification Transformer whose output is realized back into valid packets. The chapter is organized as the following. Section 4.1 fixes the threat model, since later decisions are justified relative to it. Section 4.2 gives the end-to-end view of the architecture and its two operating phases. Section 4.3 describes how the training corpus is constructed and how raw captures become the tensors the models consume. Sections 4.4 and 4.5 then detail the two functional stages, detection and purification, and Section 4.6 closes with the implementation. Design

decisions and their rationale belong to this chapter; the measured behaviour of the resulting system is the subject of Chapter 5.

4.1 Threat Model

The scenario, illustrated in Figure 4.1, involves three parties. A protected NIDS guards a network and is assumed to handle conventional, non-adversarial attack traffic; this is the detector an adversary wants to get past. An attacker manipulates the traffic of an ongoing attack so that this NIDS no longer recognizes it, in the shape of the evasion attacks of Section 2.2. DEPT sits inline, immediately upstream of the protected NIDS: its job is to decide, packet by packet, whether a flow has been adversarially manipulated, and to neutralize the manipulation before the protected system ever consumes the flow.

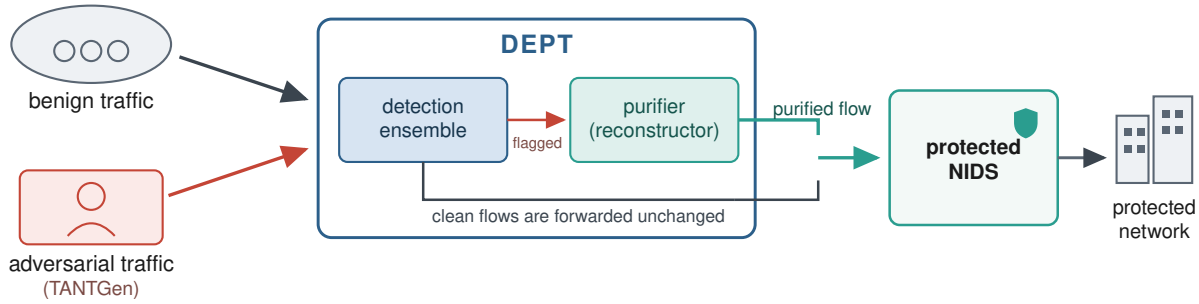


Figure 4.1: DEPT's deployment context. Benign traffic and adversarially manipulated traffic (generated by TANTGen in this work) arrive interleaved. DEPT inspects every flow in streaming fashion: flows judged clean are forwarded unchanged, while flagged flows pass through the purifier first, so that the protected NIDS downstream receives traffic whose adversarial manipulation has been undone.

The attacker is TANTGen [13], the Transformer-based adversarial traffic generator developed in the sibling thesis and reviewed in more depth in Chapter 3. This choice pins the threat model to capabilities that have been demonstrated rather than assumed. The attacker operates in the black-box setting of Section 2.2.1: it has no access to the architecture, parameters, or gradients of DEPT or of the protected NIDS, and can at most observe detection outcomes while probing. It perturbs live attack traffic along two physical axes, the timing of packets (their inter-arrival times) and their size (payload padding), which TANTGen exposes as three perturbation modes: `iat` (timing only), `size` (size only), and `both`. Crucially, the generated traffic is protocol-compliant by construction; it consists of packets that can actually be sent and that a dissector parses without complaint.

We further adopt a functional realizability assumption: attacks that violate protocol semantics, for instance a broken checksum, a length field inconsistent with the bytes, or a negative inter-arrival time, are out of scope by design. This is a scoping decision rather than a weakness. As discussed in Section 2.2.3, a perturbation that describes no sendable packet is not a working evasion in the first place, and such malformed traffic is trivially rejected by stateless checks well before any learned model needs to be

involved.

TANTGen fixes the threat model to the demonstrated capabilities, but it also sets an upper-bound on what that model can be used to prove. Being gradient-free, it cannot showcase a gradient path, and a central claim of this design, that fusing a non-differentiable learner with a differentiable one raises the cost of evasion, is a claim precisely about gradient paths. Establishing it needs an adversary that has one.

We therefore admit a second, strictly stronger adversary alongside TANTGen, and evaluate against it in Section 5.7. It is granted white-box access to the TCN: exact architecture, weights and gradients. It is granted nothing at all for the Random Forest, which is not merely withheld but non-differentiable, so no gradient exists to leak. This is the sharpest form of the heterogeneity question rather than a concession: the attacker holds one half of the ensemble exactly and must reach the other half through the traffic they share. Its perturbations obey the same functional realizability assumption as TANTGen's and are further constrained to be increase-only, since packets already on the wire may be padded or delayed but never un-sent, and are measured in the units an attacker actually spends, bytes of padding and seconds of delay.

Three non-goals complete the model. First, DEPT does not defend against poisoning: its training data is curated offline from known sources, and the training pipeline is trusted. Second, no claim is made against an adversary with gradient access to every component, which for the Random Forest is not a realistic capability but an impossible one. Third, DEPT does not replace the protected NIDS. Detecting the underlying attacks themselves (a flood, a scan, a brute-force attempt) remains the downstream system's job; DEPT answers only the narrower question of whether a flow has been adversarially tampered with. This division of labour directly shapes the labelling convention of Section 4.3.

4.2 System Overview

Figure 4.2 shows the complete architecture. Traffic enters through the Packet-to-Tensor pipeline, which converts each flow into the masked nPrint representation of Section 2.1.2 and emits it at two rates: a dense per-packet token stream and a sparse stream of prefix statistics. The two base learners consume one stream each. The TCN produces a verdict logit at every packet, from the very first one; the RF produces a logit at packet 2 and then every tenth packet, on the aggregated prefix. A per-packet fusion token joins the two prediction streams with a small amount of context, and the meta-learner, a compact causal Transformer, turns that token stream into a per-packet probability that the flow is adversarial. Flows that never cross the decision threshold are forwarded untouched. Flows that do are handed to the reconstructor, a causal purification Transformer that undoes the manipulation in feature space, and a realization step then writes that correction back onto the packets themselves. Realization is the last

stage of the inference path: what reaches the protected NIDS is the realized traffic, not the purified tensors.

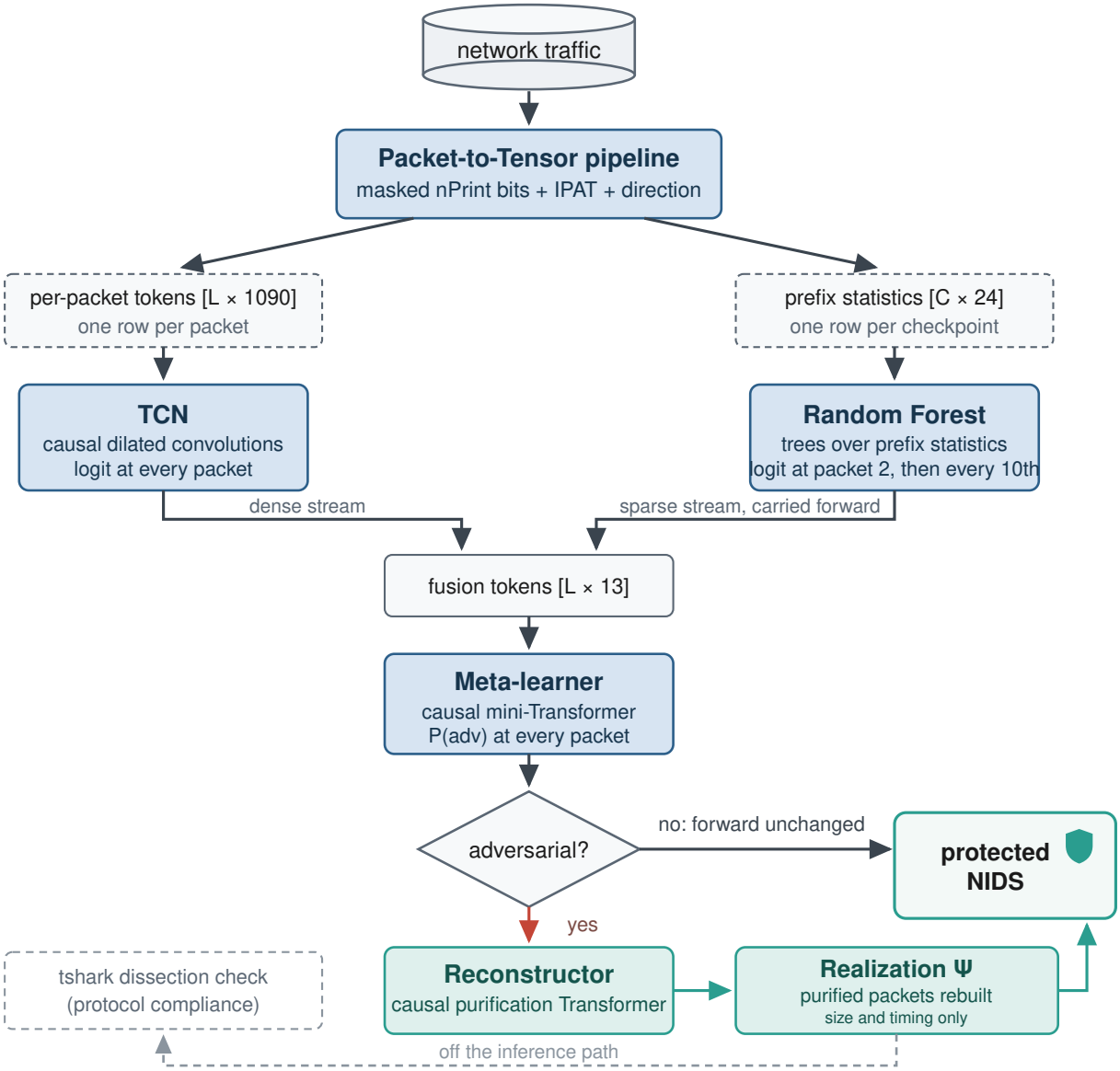


Figure 4.2: The DEPT architecture. The Packet-to-Tensor pipeline feeds two representations of the same flow to two architecturally different base learners; their prediction streams are joined into 13-dimensional per-packet fusion tokens and fused by a causal mini-Transformer into a streaming verdict. Only flagged flows pay the cost of purification.

The shape of the system follows from the objectives stated in Section 1.3. The shared representation and its dual emission cadence answer O1: both learners see the same masked, behaviour-focused view of a flow, each at the granularity it is suited for. The ensemble answers O2: the two base learners differ in architecture (non-differentiable trees against a differentiable network), in input (aggregated statistics against raw sequences), and in timing (sparse against dense), which is exactly the diversity that the

transferability argument of Section 2.3 asks for. The reconstructor and the realization step together answer O3, restoring flagged flows to the benign manifold and handing them on as sendable packets. The streaming nature that defines all of it, verdicts at every packet rather than at end of flow, is what O4's evaluation axis is later measured against.

DEPT operates in two phases. In the training phase, all components are fitted offline, in dependency order. The pipeline first tensorizes every source once. The two base learners are then trained independently on the training split. The meta-learner is trained next, not on the training split but on the validation split, where the base learners' predictions are out-of-fold — FLAG why does this matter?; Section 4.4.3 returns to why this matters. The reconstructor is trained last, on matched pairs of adversarial and clean flows. Its purification effect is then quantified primarily against the independent NIDS each attack was built to evade, Kitsune or TANTGen's ML-NIDS (Section 5.8), with the already-trained TCN kept only as a secondary check; Section 5.6 discusses why the external judge, not DEPT's own detector, carries the end-to-end claim. In the deployment phase the flow of Figure 4.2 runs continuously: tensorize each arriving packet, run through both base learners, fuse, and decide. Purification is gated on the ensemble's flag, so the common case, clean traffic, never pays for it, and only the flows the detector marks adversarial are handed to the reconstructor and then realized back into packets before reaching the protected NIDS. This detect-then-purify cycle, realization included, is the design point on which the real-time requirement of Section 1.2 rests; measuring it in wall-clock terms is left to future work (Section 6.4).

4.3 Dataset Construction and Packet-to-Tensor Pipeline

A – Traffic corpus. The corpus pairs ten benign traffic sources with adversarial counterparts generated by TANTGen. Five sources carry over from the datasets most used in the related literature: the Mirai botnet and SSDP flood captures of Kitsune [16], and the Distributed Denial of Service (DDoS), port scan, and FTP/SSH brute-force captures of CIC-IDS2017 [17]. Five more come from CIC-DDoS-2019's reflection and amplification floods (NTP, DNS, SSDP, SYN, and LDAP) [18]. For each source, TANTGen produced one adversarial variant per perturbation mode (`both`, `iat`, `size`), giving thirty adversarial source–mode combinations. Two construction decisions deserve mention. The CIC-DDoS-2019 captures are attack-dominated from their first seconds and offer no clean time window to filter on, so each is truncated to its first million packets instead, the same scale at which the generator caps its own malicious input. And no source without a TANTGen counterpart was admitted into the corpus: benign flows from unpaired captures would inflate the benign class without contributing any adversarial signal to learn against.

B – Label convention. Every flow taken from the raw datasets is labelled benign, including their native attacks, and every TANTGen-generated flow is labelled adversarial. This follows directly from the threat model: detecting a flood or a scan is the protected NIDS's responsibility, while DEPT's task is to recognize whether a flow has been manipulated. The labels therefore encode precisely the question the system must answer, "did the generator touch this flow". The convention has a measurable consequence, namely that a source whose features TANTGen barely moves is close to unlabellable no matter the detector, and Section 5.9 analyses this cost openly.

C – From packets to tensors. Each capture is processed in a single pass. Packets are grouped into flows by their canonical five-tuple and kept in timestamp order, and each packet is encoded into the 1090-dimensional token of Section 2.1.2: the 1088 masked nPrint header bits plus the z-scored IPAT and a direction bit (the normalization statistics are computed at tensorization time and stored alongside the tensors, so that training and inference standardize identically). The per-flow token sequences are stored as one large ragged array with an offsets table, which keeps loading cheap and lets every downstream model share the same on-disk artifacts.

The same pass also produces the sparse representation for the RF. At each checkpoint, six base features (inter-arrival time, packet size, payload length, payload entropy, TCP flags, and direction) are summarized by their mean, standard deviation, minimum, and maximum over the whole prefix seen so far, a 24-dimensional vector. These statistics are defined incrementally, by Welford's online algorithm for means and variances together with running extremes, so that a flow never needs to be re-buffered: the cost per packet is constant, and the statistics available at a checkpoint are exactly those the forest would see at the same point of a live flow. Offline the whole capture is already in memory, so the pipeline takes a shortcut and computes the same statistics with vectorized per-flow cumulative sums, which is much faster and gives the same values. The incremental definition is still the one that matters in deployment, where a given input has only one packet at a time to work with. Checkpoints fire at packet 2 and then every tenth packet, the cadence motivated in Section 2.1.2: without the early checkpoint, flows shorter than ten packets, which include essentially every port-scan flow, would end before the forest ever voted. Figure 4.3 lays the resulting prediction pattern out on the packet axis, including the carry-forward behaviour that the fusion stage of Section 4.4.3 builds on.

D – Splits and class balance. All models share a single $70/15/15$ train/validation/test split, drawn with seed 42. The split is flow-atomic: every row derived from a flow, be it a packet token, a checkpoint vector, or a reconstruction pair, lands on the same side, since splitting per row would leak near-duplicate prefixes of the same flow across sets. It is also stratified by source, so each split preserves the corpus's source mixture, and it is reused verbatim by the RF, the TCN, the meta-learner, and the reconstructor. This shared split is what later makes their comparison like-for-like, and it guarantees that the meta-

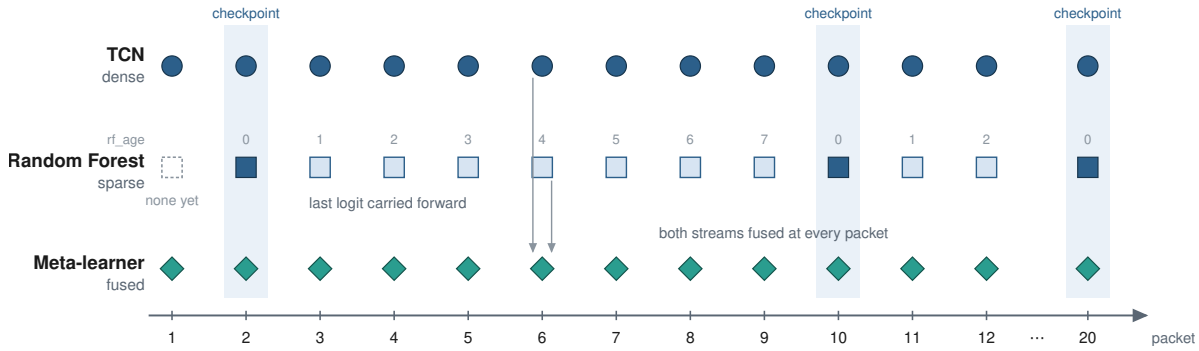


Figure 4.3: The dual prediction cadence over the first packets of a flow. The TCN (circles) emits a verdict at every packet, starting at the first. The Random Forest (squares) emits only at prefix checkpoints (packet 2, then every tenth packet, shaded); between checkpoints its last logit is carried forward, and before packet 2 it has no verdict at all. The `rf_age` row shows the recency counter that the fusion token exposes to the meta-learner. The meta-learner (diamonds) fuses both streams into a fresh verdict at every packet.

learner, which consumes the base learners' outputs, is never evaluated on flows any base learner trained on. Class imbalance is treated at two levels. Because the reflection floods contribute vastly more benign flows than the other sources, the benign pool is capped at three benign flows per adversarial flow before training; the residual imbalance is then handled by the learners themselves, with balanced class weighting in both the forest and the neural losses.

4.4 Primary Detection: Heterogeneous Stacking Ensemble

The detection stage instantiates the stacking pattern of Section 2.3: two deliberately dissimilar base learners and a learned combiner. The base learners are described first, the fusion afterwards.

4.4.1 Random Forest Base Learner

The RF consumes the 24-dimensional checkpoint vectors and answers, at every checkpoint, whether the prefix statistics look manipulated. The choice of a forest for this role is doubly motivated. Practically, forests are strong learners for small tabular inputs and don't require neither feature scaling nor much tuning (Section 2.1.3). Strategically, a forest's decision function is piecewise constant and non-differentiable, which makes it the gradient-free member that the transferability argument of Section 2.3.2 requires: even a surrogate-based attacker obtains no smooth loss surface that comes from the forest's verdicts.

The configuration was selected by grid search on the validation split, over $\{200, 400\}$ trees, maximum depths $\{\text{unbounded}, 20, 40\}$, and minimum leaf sizes $\{1, 2, 5\}$, with $\sqrt{d}$ feature sampling and the `balanced_subsample` class weighting fixed throughout. The selected model uses 200 trees of maximum

depth 20 with at least 5 samples per leaf. It is worth noting that the search favoured the most regularized corner of the grid: with only 24 input dimensions, deeper, unconstrained trees mostly memorize checkpoint idiosyncrasies, while a bounded depth and a non-trivial leaf size force better generalization across flows. At inference time the forest is queried exactly at the pipeline's checkpoints, so its verdict stream is sparse, delayed by design, and at the cadence of Figure 4.3.

4.4.2 Temporal Convolutional Network Base Learner

The TCN is the dense half of the ensemble. It consumes the full per-packet token stream and emits a logit pair at every position, so a flow acquires its first verdict with its first packet. The network instantiates the architecture of Section 2.1.3: three residual temporal blocks of 128 channels each, kernel size 3, dilations 1, 2, and 4, and dropout of 0.15, closed by a per-timestep linear head; in total about 0.8 million parameters. Causal padding keeps every output independent of future packets, which is the property that makes the model legitimate for streaming use, and the dilation schedule gives the receptive field of 29 packets derived in Equation (2.1), enough context to cover several RF checkpoint intervals (of roughly 10) without inflating the parameter count.

Training minimizes per-packet cross-entropy with balanced class weights, averaged within each flow before averaging across flows, so that a ten-thousand-packet flood does not drown out two-packet scans in the gradient. Sequences are truncated at 200 packets when batching, so that a single long flow does not force the whole padded batch to its length; truncation therefore discards supervision beyond packet 200 rather than distorting it, which is consistent with a wish for early-detection. Optimization uses AdamW with a cosine learning-rate schedule and early stopping on the validation split. The intended division of labour inside the ensemble is thus clear-cut: the TCN contributes an early, dense, but noisier signal, the forest a delayed, sparse, more accurate one, and neither is expected to be sufficient alone.

4.4.3 Meta-Learner (the "Mini-Transformer")

The meta-learner reconciles the two prediction streams. The difficulty is not accuracy but asynchrony: at most packets the forest has no fresh opinion, and before packet 2 it has none at all, so a fixed rule such as averaging is not even well-defined at every position. DEPT's answer is a small decoder-only Transformer over per-packet fusion tokens, whose 13 dimensions are listed in Table 4.1. The token gives the model the two base logits, the information needed to interpret them (how stale the forest's logit is, whether it was refreshed at this very packet), and a deliberately compact summary of the current packet.

Three design decisions define this component. First, the input is kept minimal on purpose: the meta-learner is a stacker, not another feature extractor, and withholding the full 1088-bit representation forces it to combine the base learners instead of bypassing them; it also keeps the model at roughly 68k

Table 4.1: The 13-dimensional per-packet fusion token consumed by the meta-learner.

Slot	Feature	Meaning
0	RF logit	last checkpoint logit, carried forward; placeholder before packet 2
1	TCN logit	dense per-packet logit
2	`rf_age`	packets since the last RF refresh, instantiated at 30
3	`is_rf_checkpoint`	1 iff the RF logit was refreshed at this packet
4	IPAT	z-scored inter-packet arrival time
5	direction	direction bit of the current packet
6	packet length	IPv4 total length, normalized by 1500
7–12	TCP flags	URG, ACK, PSH, RST, SYN, FIN bits

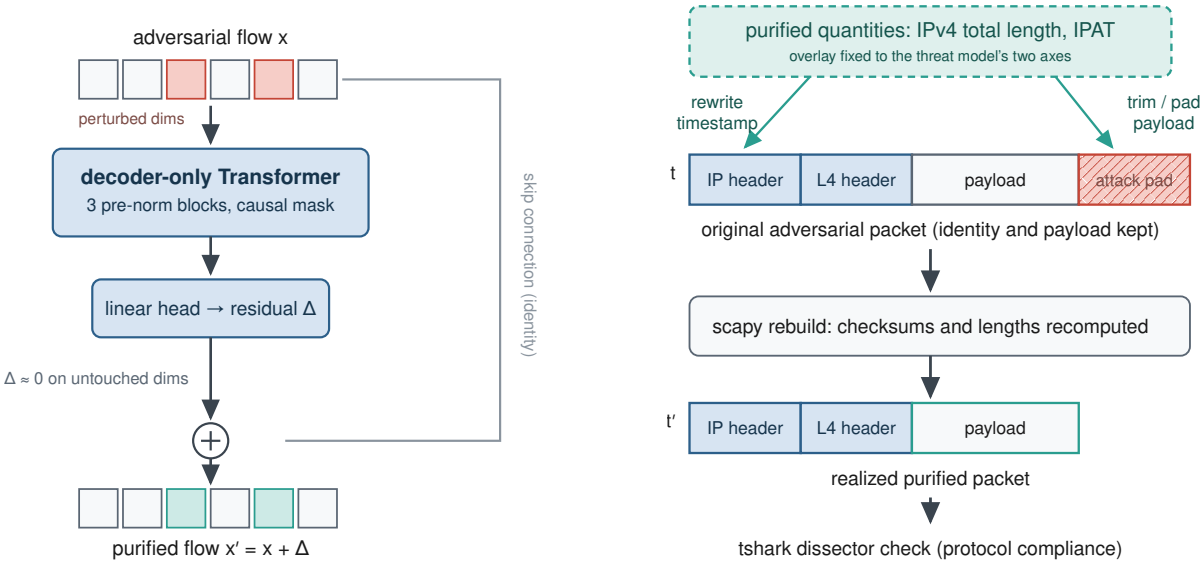
parameters (two causal blocks, four heads, $d_{\text{model}} = 64$), negligible next to either base. Second, the pre-checkpoint gap is filled by a learnable placeholder logit rather than an imputed constant. A zero would be read as a genuine "undecided" forest opinion; a trainable parameter lets the model learn its own prior for "the forest has not spoken yet". Third, attention is causally masked and positions are encoded with the standard sinusoidal scheme, so the fused verdict at packet t depends only on packets up to t, preserving the streaming property end to end.

The training discipline matters as much as the architecture. Fitting a stacker on the same data its base learners trained on would let it learn their overfit confidence rather than their generalizing behaviour [14], so the meta-learner trains on the validation split, where both base learners' predictions are out-of-fold, and that split is sub-divided (seed 43) into a meta-train and a meta-val part, the latter using early stopping on streaming Area Under the Receiver Operating Characteristic curve (AUROC) at packet 10. The loss is the same balanced per-packet cross-entropy as the TCN's, plus an auxiliary "beat the best base" term (weight 0.5) that penalizes, at each packet, any shortfall of the meta-learner's true-class confidence against the better of the two base learners. The intent is to make "never worse than the best base" an explicit training signal; what this loss does and does not achieve in practice is reported with the ablations in Section 5.5. At deployment the meta-learner's per-packet probability is compared against a fixed threshold of 0.5; a flow is flagged the first time its score crosses it, which is also the event the earliest-detection metric of Chapter 5 records.

4.5 Input Purification: Reconstructor

Detection alone would leave only a blunt response to a flagged flow, dropping it, which turns every false positive into a denial of service for a legitimate client. DEPT's second stage takes the purification route of Section 2.2.2 instead: undo the manipulation and let the repaired flow proceed. The reconstructor is a causal Transformer denoiser operating directly on the per-packet token sequence, trained to map an adversarial flow onto its clean counterpart (Figure 4.4). It is therefore not a generative purifier in the

sense of Section 2.2.2: it models no distribution over benign traffic and samples nothing, but regresses the correction directly.



(a) Feature-space purification: a decoder-only Transformer predicts a per-dimension residual correction Δ, and the purified flow is $x+\Delta$.

(b) Traffic-space realization: the purified length and timing are written back onto the original packets, and the result is checked with a dissector.

Figure 4.4: The purification stage. Purification happens in feature space (a), over all 1090 dimensions and with no knowledge of which the attacker moved; realization (b) then overlays the two purified quantities onto the original adversarial packets, preserving identity and payload content, and recomputes the dependent header fields. The packets leaving (b) are what the protected NIDS receives. Only (a) is attack-agnostic: the overlay in (b) is fixed to this threat model's two axes (Section 4.5.3).

4.5.1 Purification Model

The reconstructor shares the meta-learner's scaffold, the decoder-only stack of Section 2.1.3: pre-norm self-attention blocks under a strict causal mask, with sinusoidal positions, scaled up to three layers, four heads, and $d_{\text{model}} = 128$ (about 678K parameters), with the 1090-dimensional token as both input and output. Its head, however, does not predict the clean sequence directly: it predicts a per-dimension residual Δ, and the output is the input plus Δ. This residual formulation encodes the right principle for the problem—an adversarial example is, by definition, a very small perturbation of a clean sample. TANTGen's manipulations are no different: they touch only two physical quantities, timing and size, so out of the 1090 dimensions of a token only a handful actually need correcting; with a residual head, the identity map is the easy default ($\Delta \approx 0$), the untouched ~ 1050 header bits are preserved, and the model's capacity is spent on the localized corrections that matter. The causal mask, in turn, keeps the purifier valid for streaming use and, as discussed previously, makes it amenable to key–value caching.

The decoder-only label is architectural, not a claim of autoregression: the purifier never consumes its own output, but corrects every input position in a single pass, each from packets $\leq t$ alone.

4.5.2 Training on Matched Flow Pairs

Purification is learned from matched pairs. Because TANTGen perturbs a flow without changing its five-tuple identity, every adversarial flow can be aligned with the exact clean flow it was derived from by matching canonical five-tuple keys; the result is a corpus of (adversarial $\rightarrow$ clean) sequence pairs on which reconstruction is supervised directly. Pairs shorter than two packets are dropped, since a single packet carries no inter-arrival interval and thus no timing signal to restore. Pairs follow the same seed-42 flow-atomic split as everything else, keeping the test pairs unseen by every base learner.

The loss is a smooth-L1 distance between the reconstructed and the clean sequence, masked over padding and averaged within each flow before averaging across flows, the same length-normalization argument as in Section 4.4.2; smooth-L1 is preferred over a plain squared error for its robustness to the occasional large deviation. One further choice deserves emphasis: the loss weights all 1090 dimensions uniformly. The training signal contains no hint of where TANTGen's perturbation lives, so the model must discover for itself which dimensions are trustworthy and which have been tampered with. This keeps the purification model attack-agnostic: nothing in its objective is specific to TANTGen's footprint (even though we know it). During development a variant weighted towards the known perturbed dimensions was also explored, and it was dropped from the final design precisely because it ties the defense to one attacker's signature. This agnosticism is a property of this feature-space stage, not of the whole purification path, and Section 4.5.3 states where it stops.

4.5.3 Traffic-Space Realization and Validation

A purifier that edits traffic inherits the realizability obligation of Section 2.2.3: its output must be traffic that can exist on the wire. DEPT assumes this obligation with an explicit realization step (Figure 4.4(b)), and places it at the end of the inference path rather than beside it: the protected NIDS sits behind DEPT and consumes traffic, not tensors, so the realized packets are the pipeline's output. Rebuilding packets from scratch out of the purified tensors would be impossible, as nPrint encodes only headers and the identity fields are masked to zero. Realization therefore works by overlay: we keep each original adversarial packet, with its identity and payload content intact, and write back only the two quantities purification may have changed. The purified IPv4 total length is decoded from its sixteen header bits and applied by trimming the application payload, which is the inverse of TANTGen's size perturbation; the purified inter-arrival time is de-normalized and applied to the packet's capture timestamp. The packets are then re-serialized with scapy, which recomputes the checksums and length fields that depend on the edit.

Alignment between tensor rows and packets reuses the pipeline's canonical flow grouping, giving a one-to-one correspondence; any flow whose packet count fails to match its tensor length is skipped and counted, not silenced.

Placing the realization step in the pipeline has a price, and it is stated plainly here: the deployed pipeline is not attack-agnostic end-to-end. The reconstructor of Section 4.5.2 corrects all 1090 dimensions without being told which ones the attacker moved; realization writes back exactly two, because those are the two axes the threat model of Section 4.1 admits. An adversary that instead perturbed, for example, the Time-to-Live (TTL), the TCP window, or the option layout would still be flagged by the ensemble and denoised in feature space, but its correction would be discarded on the way back to the wire, since no overlay rule exists for those fields; the flow would reach the protected NIDS unrepaired, and DEPT would degrade to detection alone. Agnosticism to what the attacker moved is therefore a property of DEPT's detection and of its denoiser, not of the full system that is actually deployed. The overlay is a per-dimension fix and generalizing it, by deriving the fields to edit from the purifier's own residual rather than assuming them in advance, is left as future work (Section 6.4).

Every realized packet is finally passed through the `tshark` dissector, and protocol compliance is scored as the fraction of packets parsed without a malformed-packet complaint. This check is the one part of the stage that stays off the inference path: it is a measurement, for the compliance figures of Section 5.6 and for forensics, not a runtime filter, so no packet waits on it. One structural caveat is worth stating already here: for application-layer text protocols, such as the SSDP `M-SEARCH` messages that dominate one source, a size edit necessarily corrupts the L7 grammar even when all IP and UDP fields remain consistent, so such packets cannot be faithfully realized by overlay at all. They are reported separately rather than scored as compliance failures, and Section 5.6 gives the full accounting.

Finally, the evaluation harness embeds the independent protected detectors that complement the TCN-based purification measurements: a Kitsune instance (KitNET with the damped incremental statistics its reference implementation calls AfterImage [16]) for the two Kitsune-native sources (Mirai and the SSDP flood), and TANTGen's own ML-NIDS flow classifiers for the eight CIC attacks, in every case the very detector TANTGen calibrated that attack's evasion against. The harness realizes matched adversarial, purified, and clean packet triplets of the same flows and scores each through its attack's detector, so that purification can be assessed by a judge that shares nothing with DEPT's training loop; the measurement is reported in Section 5.8.1.

4.6 Implementation and Tools

DEPT is implemented in Python. The three sequence models (TCN, meta-learner, reconstructor) are built in PyTorch and the RF in scikit-learn. The nPrint encoding is reimplemented natively on top of scapy,

following the bit layout of the original tool [22]; writting the encoding into the pipeline avoids an external binary, lets identity masking happen inline, and keeps tensorization a single pass over each capture. Scapy also drives packet parsing and the realization step, while `tshark`, `editcap`, and `tcpdump` handle dissection, time-window filtering of CIC-IDS2017, and the packet-capping of the large CIC-DDoS-2019 captures, respectively.

Table 4.2: Configuration summary of DEPT's four learned components. A step is a checkpoint for the Random Forest and a packet for the three sequence models; the forest additionally uses $\sqrt{d}$ feature sampling and `balanced_subsample` class weighting.

	RF	**TCN**	**Meta-learner**	**Reconstructor**
Input (per step)	24	1090	13	1090
Blocks / layers / trees	200 trees	3 temporal	2 causal	3 causal
Width / depth	max depth 20	128 channels	$d_{\text{model}} = 64$	$d_{\text{model}} = 128$
Attention heads	—	—	4	4
Feed-forward dim	—	—	128	256
Regularization	min leaf 5	dropout 0.15	dropout 0.1	dropout 0.1
Parameters	—	805,762	67,971	677,698
Loss / criterion	Gini impurity	balanced CE	balanced CE + aux	smooth-L1
Learning rate	—	10^{-3}	$3 \cdot 10^{-4}$	$3 \cdot 10^{-4}$
Batch size	—	32	128	32
Max epochs	—	20	20	20
Selection criterion	val macro-F_1	val macro-F_1	AUROC@10	val loss

The system is organized as four model stages behind five configuration files, one per stage plus one for the pipeline, with the two evaluation studies of Chapter 5 (the deployment baseline and the gradient ablation) adding one configuration file each in the same pattern; every file fully determines its stage's data scope, architecture, and training settings, and a small runner script per stage executes it end to end. All stages read the same tensor artifacts and the same stored split, which is how the shared-split discipline of Section 4.3 is enforced in practice, and derived artifacts are cached and fingerprinted against their upstream inputs (for instance, the meta-learner's token cache is invalidated whenever a base learner's checkpoint changes), so any partial re-run stays consistent. Table 4.2 summarizes all four learned components. The source layout mirrors the stages, with the feature pipeline under `src/feature_extraction`, the models under `src/models`, per-stage entry points under `scripts`, and configuration under `config`; further operational details live within the github repository rather than in this document.

With the architecture in place, what remains is to show that it meets its goals: that the ensemble detects manipulation early, and that purification actually returns flagged flows to the benign manifold while emitting valid packets. These are the questions Chapter 5 takes up.

5

Evaluation

Contents

5.1 Research Questions 44
5.2 Experimental Setup 44
5.3 Metrics 46
5.4 Base Learner Results 47
5.5 Meta-Learner Results 49
5.6 Purification Results 51
5.7 Ensemble Robustness under a Gradient-guided Attacker (RQ1) 55
5.8 End-to-End System Evaluation 59
5.9 Discussion and Limitations 64

This chapter evaluates DEPT against the threat model of Section 4.1: a black-box, protocol-compliant adversary, concretely the TANTGen generator of Section 4.3 treated as the red team, that manipulates a streaming flow so as to evade a downstream NIDS. The chapter first fixes the research questions (Section 5.1), the corpus and environment (Section 5.2), and the metric definitions (Section 5.3). It then reports the two base learners (Section 5.4), their attention-based fusion (Section 5.5), and the purification stage (Section 5.6), before turning to the end-to-end system (Section 5.8) and to an explicit

accounting of the limitations (Section 5.9). Throughout, the results are reported as measured, including the sources and perturbation modes on which DEPT performs poorly.

5.1 Research Questions

The evaluation is organized around two research questions, each tied to one of the work objectives of Section 1.3:

- **RQ1 (ensemble robustness).** Does heterogeneous stacking, a non-differentiable Random Forest fused with a differentiable Temporal Convolutional Network, measurably raise the cost of a transferable, gradient-guided evasion attack relative to a single-architecture baseline?
- **RQ2 (purification effectiveness).** Does the reconstructor move a flagged adversarial flow back onto the benign manifold, closing the detection gap seen by the protected NIDS, while emitting a realizable, protocol-compliant packet sequence?

Both questions are answered by measurement. RQ2 is answered through the reconstructor's own metrics (Section 5.6) and, end to end, through the deployment baseline that scores the defense against the independent NIDS each attack was built to evade (Section 5.8).

RQ1 needs an adversary TANTGen cannot supply. TANTGen is black-box and gradient-free by design, so it leverages no gradient path and cannot say what heterogeneity is worth against it. Section 5.7 therefore introduces a second, complementary adversary that differentiates the TCN directly and whose perturbation is then scored, unchanged, against the Random Forest, the TCN, and the fused stack. The asymmetry is the point: the attacker has an exact gradient into one half of the ensemble and none into the other, which is the situation RQ1 describes.

A third natural question, whether the detect-then-purify pipeline stays within a real-time inference budget, is deliberately not addressed here: the wall-clock latency and overhead evaluation of the delivered system is left to future work (Chapter 6).

5.2 Experimental Setup

5.2.1 Datasets and Threat Generation

The evaluation corpus combines ten benign traffic sources (Mirai and the SSDP flood from Kitsune; DDoS, Port Scan and Brute Force from CIC-IDS2017; and the five CIC-DDoS-2019 reflection and amplification floods, namely NTP, DNS, SSDP, SYN, and LDAP), each paired with three TANTGen adversarial variants: `both` (timing and size), `iat` (inter-arrival timing only), and `size` (payload/size only). The

per-source detection tables of this chapter therefore cover ten sources and thirty source–mode combinations. Under DEPT's label convention (Section 4.3), every raw-capture flow is labelled benign, since the upstream NIDS is responsible for the datasets' native attacks, and every TANTGen-generated flow is labelled adversarial. The task DEPT is measured on is therefore precise: distinguish a flow that TANTGen has manipulated from one it has not, and undo that manipulation. This framing is a deliberate choice, and its consequences (in particular, that a source on which TANTGen barely perturbs the features is essentially unlabelable) are analysed in Section 5.9.

The corpus as tensorized comprises $8{,}624{,}958$ flows and $61{,}516{,}406$ packets across the forty sources (ten benign and thirty adversarial). All models share a single flow-atomic, source-stratified $70/15/15$ split (seed 42), so that the RF, TCN, meta-learner, and reconstructor are trained, validated, and tested on the same flows and their results are directly comparable. Because the CIC-DDoS-2019 reflection floods contribute the overwhelming majority of adversarial flows, the held-out test pool is adversarial-dominated: of the $1{,}293{,}737$ test flows seen by the TCN and meta-learner, $1{,}082{,}586$ (83.7%) are adversarial and $211{,}151$ are benign. The RF, whose first checkpoint fires at packet 2, never sees the $6{,}179$ single-packet flows and therefore scores $1{,}287{,}558$ flows on the streaming axis, or $1{,}838{,}798$ rows on its native per-checkpoint axis. This base rate matters when reading accuracy and precision, and it is the reason the cross-model comparison is anchored on base-rate-robust quantities (AUROC and PR AUC) and on the streaming axis rather than on raw accuracy.

5.2.2 Implementation Environment

All training and evaluation were run on a single research server: $2\times$ Intel Xeon Gold 5320 CPUs @ 2.2 GHz (52 physical cores, 104 hardware threads), 160 GB of RAM, and $2\times$ NVIDIA RTX 6000 Ada GPUs. The software stack is the one described in Section 4.6: PyTorch for the TCN, meta-learner, and reconstructor; scikit-learn for the Random Forest; and Scapy together with `tshark`/`editcap`/`tcpdump` for PCAP handling, feature realization, and protocol-compliance checking.

The full pipeline is expensive to run end to end, and the cost is worth recording since it bounds how much exploration a project of this size can afford. Generating the thirty adversarial captures with TANTGen took 26.7 h, dominated by the NTP flood alone (11.9 h), and tensorizing the forty sources took a further 11.2 h. Of the model stages, the RF completed its eighteen-point grid search and final fit in 2.7 h on CPU; the TCN trained for 24.7 h, early-stopping after ten of twenty planned epochs with the best epoch at 6; the meta-learner for 3.4 h, best epoch 17 of 20; and the reconstructor for 10 h, best epoch 19 of 20. The two comparison studies of Section 5.7 and Section 5.8 cost 4.3 h and 33.6 h respectively, the latter because it scores $2{,}704{,}816$ realized flows through the independent detectors. Repeating the four model stages once per attack, as Section 5.8.2 does, adds roughly 28 h more.

5.3 Metrics

The metrics separate into three families.

A – Static classification. Treating each scored unit independently at the end-of-flow (or per-checkpoint) verdict, the evaluation reports ROC AUC and PR AUC (threshold-free ranking quality; PR AUC is the more informative summary under the $\approx 83\%$ adversarial base rate), Macro F1 (the unweighted mean of the benign and adversarial F_1 at threshold 0.5), and the per-class precision and recall. Recall on the adversarial class is reported separately, because a detector that misses adversarial flows is failing at the one task it exists for.

B – Streaming. A defense that must act before a flow completes cannot be judged only at end-of-flow. The evaluation therefore reports AUROC@t: the ROC AUC computed using, for each flow, only the prediction available after its first t packets. This exposes how quickly a model becomes discriminative. It is paired with the Earliest Detection Packet (EDP), the median packet index at which a flow's streaming score first crosses 0.5, reported with its inter-quartile range, and with the detection rate, the fraction of adversarial flows that ever cross the threshold. AUROC@t measures ranking quality; EDP and detection rate measure operational timeliness.

C – Purification. The reconstructor is judged on four quantities. Reconstruction Mean Squared Error (MSE) is computed on the feature dimensions TANTGen actually perturbed, between the flow and its true clean counterpart, measured before (adversarial input) and after (reconstructor output). The headline efficacy quantity is detection restoration through an independent NIDS: the clean, adversarial, and purified (or DEPT-defended) flows are scored through the real NIDS each attack was built to evade, Kitsune or the ML-NIDS, and the restoration ratio, the fraction of the adversarial-to-clean gap recovered, together with the Attack Success Rate (ASR) before and after DEPT, report how much detection the defense returns (Section 5.8). A secondary, in-distribution diagnostic is the purification gain, the drop in DEPT's own TCN's adversarial-probability $\Delta P(\text{adv})$ from the adversarial to the reconstructed flow, with its residual gap to a genuinely clean flow; this is circular by design (the same detector the loop trained around) and is read mostly as a sanity check, not as the end-to-end claim. Finally, protocol compliance is the fraction of realized packets that `tshark` parses without a malformed-packet error, computed only over packets the realization step can faithfully emit.

Table 5.1: Static classification results on the held-out test set. The Random Forest is scored per checkpoint ($1{,}838{,}798$ units); the TCN and meta-learner are scored once per flow at the last timestep ($1{,}293{,}737$ flows). These columns are not directly comparable across the RF / (TCN, Meta) boundary; see the streaming axis in Table 5.3.

Metric	RF (per-ckpt)	TCN (per-flow)	Meta (per-flow)
ROC AUC	**0.9852**	0.8386	0.9334
PR AUC	**0.9948**	0.9592	0.9866
Macro F_1	**0.8935**	0.6608	0.6988
Accuracy	**0.9099**	0.7621	0.7544
Precision (adv.)	0.9939	0.9220	**0.9985**
Recall (adv.)	**0.8832**	0.7820	0.7076
F_1 (adv.)	**0.9353**	0.8462	0.8282
Recall (benign)	**0.9847**	0.6606	0.9947

5.4 Base Learner Results

Table 5.1 reports the static classification results for the two base learners and the meta-learner. Two population differences must be kept in mind when reading it. First, the Random Forest is scored on its native per-checkpoint population, one row per prefix checkpoint (packet 2, then every tenth packet), giving $1{,}838{,}798$ scored units, whereas the TCN and meta-learner are scored once per flow at the last timestep. The two are therefore not directly comparable on this table; the like-for-like comparison is the streaming axis of Section 5.5. Second, all figures are computed on the same adversarial-dominated test pool, so the high adversarial recall and PR AUC partly reflect that base rate.

All three models are precision-first at threshold 0.5, and the differences between them are differences of recall. The Random Forest flags 88.3% of adversarial checkpoints while misclassifying only 1.5% of benign ones. The meta-learner is the most conservative of the three, with the highest adversarial precision (0.9985: just $1{,}115$ of $211{,}151$ benign test flows are flagged) but the lowest adversarial recall (0.7076). The TCN is the weakest on both sides, and notably weak on the benign class (0.6606 recall), which is what drags its macro-F_1 to 0.6608 despite a respectable 0.9592 PR AUC. Neither ordering should be over-interpreted from this table alone, because the RF's lead is measured on the checkpoint population that most favours it, and because macro-F_1 at a fixed threshold may confuse ranking quality with calibration.

The more informative view is per source and per perturbation mode. Table 5.2 gives the end-of-flow streaming AUROC@MAX for all 30 source–mode combinations, and Figure 5.1 plots the same values as one mark per combination. Both are computed against same-source benign flows, which is the axis that matches DEPT's task as posed: separate a manipulated flow of a given source from an unmanipulated flow of that same source, rather than from generic benign traffic drawn from the other nine. It is also the strictest of the three axes the harness records, and the one on which the system's real failure becomes visible.

Table 5.2: End-of-flow streaming AUROC@MAX per source and perturbation mode, measured against same-source benign flows. n_{adv} is the number of adversarial test flows per mode variant, as seen by the TCN and meta-learner; the RF sees the same counts except on `mirai`, where they score $1{,}957$ flows against the forest's 867, because `mirai`'s many very short flows end before the forest's packet-2 checkpoint. Values below 0.55, at or under chance, are marked with †. `b`/`i`/`s` denote the `both`/`iat`/`size` modes.

Source	n_{adv}	M.	RF	TCN	Meta
		b	0.974	0.978	0.981
`brute_force`	$1{,}045$	i	0.974	0.978	0.979
		s	0.990	0.978	0.989
		b	1.000	1.000	1.000
`ddos`	$5{,}781$	i	0.999	0.924	0.967
		s	1.000	1.000	1.000
		b	0.999	0.552	1.000
`dns_ddos`	$1{,}019$	i	0.999	0.547†	1.000
		s	0.258†	0.531†	0.443†
		b	1.000	0.500†	1.000
`ldap_ddos`	$74{,}710$	i	1.000	0.500†	1.000
		s	0.476†	0.501†	0.476†
		b	0.926	0.680	0.724
`mirai`	$1{,}957$	i	0.919	0.680	0.719
		s	0.829	0.723	0.734

Source	n_{adv}	M.	RF	TCN	Meta
		b	0.999	0.514†	0.654
`ntp_ddos`	$2{,}103$	i	0.999	0.497†	0.653
		s	0.213†	0.494†	0.493†
		b	1.000	0.997	0.998
`portscan`	$23{,}800$	i	1.000	0.652	0.757
		s	0.998	0.997	0.997
		b	1.000	0.499†	0.927
`ssdp_ddos`	$40{,}446$	i	1.000	0.502†	0.927
		s	0.409†	0.497†	0.497†
		b	1.000	0.999	1.000
`ssdp_flood`	$9{,}830$	i	1.000	1.000	1.000
		s	1.000	0.999	1.000
		b	1.000	1.000	1.000
`syn_ddos`	$200{,}483$	i	1.000	0.647	0.806
		s	1.000	1.000	1.000

The distribution is clearly not unimodal. Most combinations are "solved": 21 of the 30 reach ≥ 0.99 for the Random Forest and 13 for the meta-learner. But a cluster of four sits below chance for every model, namely the `size` variants of the DNS, NTP, SSDP, and LDAP reflection floods, where the RF scores 0.214–0.476 and the meta-learner 0.443–0.497. Below chance is a strong statement and it should be read literally: on those source–mode pairs a TANTGen-edited flow looks more like generic benign traffic than that flood's own unedited flows do, so a detector that inverted its verdict there would do better than the one that was trained. Section 5.9 discusses what this means. Two reading caveats: the per-source figures for the reflection floods rest on the RF's aggregate prefix statistics rather than on any single feature, and the `brute_force` rows, at $1{,}045$ adversarial test flows per mode, are the thinnest in the table.

Three pattern regularities emerge, and they are not the ones the design anticipated. First, the `iat`-only mode is harder for the TCN and easier for the forest: `portscan_iat` (TCN 0.652 against RF 1.000), `syn_ddos_iat` (0.647 against 1.000), `ddos_iat` (0.924 against 0.999). Aggregated prefix timing statistics capture a pure timing shift that the per-packet sequence view does not, which is exactly the architectural heterogeneity the ensemble was built on, though in the opposite direction to the one originally expected. Second, the TCN is not merely weak but blind on the four reflection floods: it sits at 0.494–0.552 on every mode of `dns_ddos`, `ldap_ddos`, `ntp_ddos`, and `ssdp_ddos`, and on `ldap_ddos`, the largest of them, it reads 0.500 to three decimals in all three modes, which is the typical of a model whose score carries no information about the label. These floods are one- and two-packet UDP exchanges, and a receptive field of 29 packets has nothing to convolve over. Third, `mirai` fails differently from the floods rather than more severely: it is the only source no model solves in any mode, its best cell reaching 0.926 where every

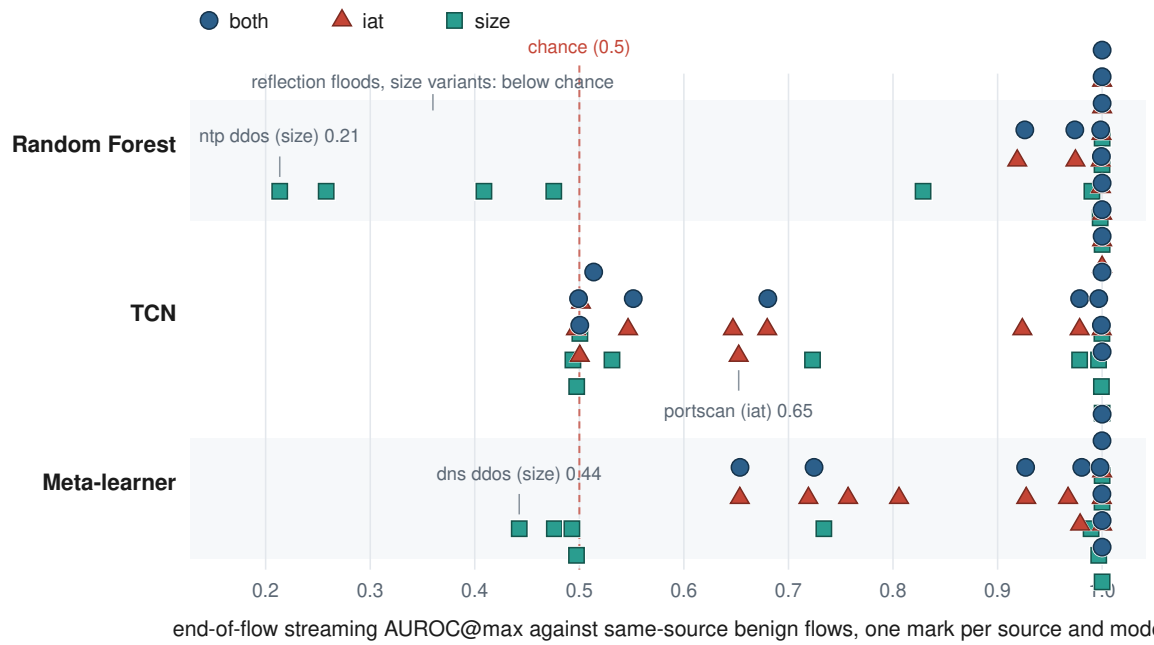


Figure 5.1: The 30 source–mode AUROC@MAX values of Table 5.2, one mark per combination and one row per model. The structure is bimodal for all three models: a dense cluster of solved combinations at ≈ 1.0, and, separated from it, the `size` variants of the four reflection floods, which fall below the chance line. The TCN's row is the widest spread of the three, reflecting that it is at chance on every reflection-flood mode.

other source reaches ≥ 0.99 somewhere, yet it never falls below chance either (0.83–0.93 RF, 0.68–0.72 TCN, 0.72–0.73 meta). It reappears as the worse source for the purifier.

5.5 Meta-Learner Results

The meta-learner is evaluated on the streaming axis, which places all three models on a common footing: each is asked, using only the first t packets of a flow, to label it as adversarial or benign. Table 5.3 reports AUROC@t for $t \in \{1, 2, 5, 10, \text{MAX}\}$ together with the median EDP and detection rate, and Figure 5.2 plots the full AUROC@t trajectories.

The result on this axis is not the one the design predicted, and it is worth stating plainly before interpreting it. The Random Forest alone is the strongest stream at every point from packet 2 onward, reaching 0.9805 at its first checkpoint and 0.9813 thereafter, against the meta-learner's 0.9099 to 0.9334 and the TCN's 0.8225 to 0.8386. The meta-learner is not the best model in this system; it is the second-best, and its advantage over the forest is confined to a single packet.

What the fusion does deliver is precise, and it is a real advantage, but it is narrower than "the stack beats its base learners". It is threefold. First, at packet 1, where the forest is silent by construction, the meta-learner has the best available verdict of any model, at 0.8373 against the TCN's 0.8191; a defense

Table 5.3: Flow-level streaming comparison (like-for-like across models). AUROC@t uses only each flow's prediction through packet t; EDP is the median packet at first threshold crossing; detection rate is the fraction of adversarial flows ever flagged. The RF issues no verdict before packet 2.

Metric	RF	TCN	Meta-Learner
First verdict at	packet 2	**packet 1**	**packet 1**
AUROC@1	—	0.8191	**0.8373**
AUROC@2	**0.9805**	0.8225	0.9099
AUROC@5	**0.9805**	0.8375	0.9319
AUROC@10	**0.9813**	0.8371	0.9323
AUROC@MAX	**0.9813**	0.8386	0.9334
EDP (median)	2 pkts	**1 pkt**	**1 pkt**
Detection rate	**88.99%**	78.40%	77.69%

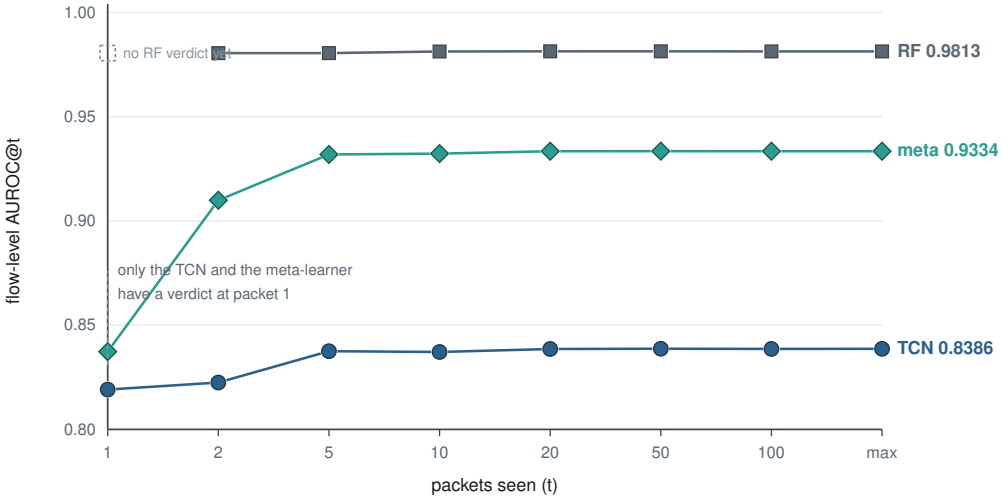


Figure 5.2: Flow-level AUROC@t as a function of the packets seen, for all measured checkpoints. The TCN and the meta-learner alone have a verdict at packet 1; the Random Forest joins at packet 2, immediately at 0.98, and stays the strongest single stream thereafter. The meta-learner rises steeply between packets 1 and 5 and then stabilizes between the two base learners.

that must act on the first packet has only these two options, and fusion is the better one. Second, the meta-learner dominates the TCN at every horizon, by 8.7 to 9.5 AUROC points from packet 2 onward, so as a way of using a sequence model it is unambiguously better than using the sequence model alone. Third, and most consequential for RQ1, the fused stack is measurably harder to evade than either component under a gradient-guided attacker, which Section 5.7 quantifies: it is the only one of the three that keeps 74.5% of flows unevaded at every budget. Robustness, not clean-traffic accuracy, is what the second base learner buys.

Two further observations complete the picture, and both cut against the fusion. The meta-learner's per-source behaviour tracks whichever base learner is stronger on that source, but not reliably: on `ssdp_ddos both/iat` it reaches 0.927 where the forest reaches 1.000, and on `ntp_ddos both/iat` it reaches 0.653–0.654 against the forest's 0.999, so on those sources it discards most of a signal it was

handed. The mechanism is the one anticipated in Section 4.4.3: the carry-forward RF logit enters the per-packet fusion token as a single, sparsely updated feature, refreshed only at checkpoints and absent before packet 2, where a learnable placeholder stands in, whereas the dense TCN logit is present and updated at every timestep, so the meta-learner's attention systematically under-weights the intermittent evidence. A stacker should be no worse than its best base learner; this one is worse on several sources and on the aggregate, and that is the clearest single target for future work on the detection stage (Section 5.9).

A note on the auxiliary "beat the best base" loss (weight 0.5, Section 4.4.3) closes this section. Over training the auxiliary term falls from 0.0267 to 0.0177, so the meta-learner does move toward matching its best base learner's confidence; over the same twenty epochs the meta-validation AUROC@10 rises from 0.9197 to 0.9333, with essentially all of that gain in the first few epochs and under 10^{-3} of movement after epoch 10. The limit seems to be ranking, not confidence: AUROC depends only on how flows are ranked by predicted score, and matching the best base learner's confidences does not reorder flows that the 13-dimensional fusion token cannot tell apart.

5.6 Purification Results

Five-tuple matching (Section 4.5.2) yields $2{,}566{,}489$ (adversarial $\rightarrow$ clean) flow pairs across the corpus, which the shared $70/15/15$ split (seed 42) divides into $1{,}796{,}542$ training pairs and a held-out $384{,}963$ test split, unseen by any base learner; the reconstructor was trained and evaluated accordingly. In feature space it is an attack-agnostic denoiser: it is trained with uniform-weight reconstruction over all feature dimensions, so it is given no hint of where TANTGen perturbed a flow. The traffic-space realization that follows is not agnostic in the same sense, being fixed to this threat model's two perturbation axes (Section 4.5.3); since it is also the stage that feeds the protected NIDS, it enforces the compliance and drives the independent-NIDS figures below, which are measured on the packets it emits. Table 5.4 summarizes the metric families overall.

The headline is the independent-NIDS row. Scored through the real detector each attack was built to evade, over $370{,}700$ realized flows, purification lifts detection from the evaded floor of 0.781 to 0.906 against a clean-oracle ceiling of 0.980, recovering 63.0% of the gap TANTGen had opened. Split by detector family the two halves behave very differently and the difference is informative: on the Kitsune-native sources detection is restored from 0.019 to 0.792 against a ceiling of 0.793, which is 99.9% of the available gap, while on the eight ML-NIDS sources it moves from 0.808 to 0.910 against a ceiling of 0.987, a 57.1% achievable improvement. The Kitsune figure is the cleaner demonstration because TANTGen had genuinely destroyed detection there; the ML-NIDS average is diluted by the many sources where detection was never lost and there was consequently little to restore.

Table 5.4: Purification results on the $384{,}963$-pair test set, overall. Reconstruction MSE is over the perturbed feature dimensions; the purification gain and residual gap are a secondary diagnostic measured through DEPT's own TCN (circular by construction, which is why the independent-NIDS restoration is the headline efficacy quantity); compliance is over realizable packets.

Family	Quantity	Value
Reconstruction	MSE on perturbed dims (before $\rightarrow$ after)	$0.918 \rightarrow \mathbf{0.054}$
	Reduction ratio	$17.0\times$
Purification (TCN)	$P(\text{adv})$: adversarial $\rightarrow$ reconstructed	$0.675 \rightarrow 0.428$
	Purification gain $\Delta P(\text{adv})$	$\mathbf{0.247}$
	Residual gap vs. clean (0.421)	0.007
Independent NIDS	Detection rate: adversarial $\rightarrow$ purified	$0.781 \rightarrow \mathbf{0.906}$
	Clean ceiling	0.980
	Detection-restoration ratio	$\mathbf{0.630}$
Compliance	`tshark` pass rate (realizable pkts)	$\mathbf{97.22}\%$ ($67{,}106/2{,}412{,}357$ bad)
	Unrealizable L7-text packets (reported apart)	$450{,}758$

The in-distribution diagnostic agrees as well: through DEPT's own TCN, the mean $P(\text{adv})$ falls from 0.675 on the adversarial flow to 0.428 on the reconstruction, against 0.421 for a genuinely clean flow, a residual gap of 0.007. This is reported as a sanity check only, since the judge belongs to the family the loop was trained on.

As with detection, the effect is strongly attack-dependent, and the per-source view is where the honest reading lives. Table 5.5 gives the rows that carry the argument, and Figure 5.3 plots every source–mode pair by its perturbation footprint (the pre-purification MSE on the perturbed dimensions) against the gain purification achieves on it. Four regimes are visible, three of them anticipated and one not.

Purification is large precisely on the structurally perturbed sources: `ddos` (gain 0.93 on `both`, 0.94 on `size`), `syn_ddos` (0.92 and 0.93), `portscan` (0.84 on both), and, in independent-NIDS terms, these are the same rows where detection is genuinely rescued: `ddos_both` from 0.080 to 0.865, `ddos_size` from 0.005 to 0.817, `portscan_both` from 0.011 to 0.679, `ssdp_flood_iat` from 0.001 to 1.000. It is correctly inert where TANTGen barely moved the features, namely the `both` and `iat` variants of the reflection floods, whose pre-purification MSE is already near zero. On the `size` variants of `ldap_ddos`, `ntp_ddos` and `ssdp_ddos` there is not even that much to undo: the attack did not fire at all there, the adversarial tensor matches the clean one cell for cell, and with no perturbed cells the footprint is undefined rather than small—the dash in Table 5.5. A gain of ≈ 0 on those rows records an attack that never happened, not a purifier that failed.

The third regime is one where purification demonstrably works but a metric cannot see it, for two different reasons. On `dns_ddos size` the footprint is maximal and substantially reduced ($1.000 \rightarrow 0.302$), yet the TCN gain stays at -0.002 because that detector assigns adversarial and clean flows the same

Table 5.5: Purification by source (representative rows). MSE on perturbed dims (before → after), purification gain $\Delta P(\text{adv})$ through DEPT's own TCN, and the independent NIDS's detection rate on the adversarial and purified streams with the clean-oracle value in parentheses. Rows are grouped by the four regimes of the text. A dash marks a quantity the run does not record for that row: no perturbed cells in the pairing, so no footprint to average over (`ldap_ddos size`), or, for `ssdp_flood size`, a stream whose packets are all L7-unrealizable.

Source	Mode	MSE (before → after)	Gain	NIDS det. adv → pur. (clean)
Footprint erased and detection genuinely restored				
`ddos`	size	$1.000 \to 0.117$	$+0.938$	$0.005 \to 0.817$ (0.828)
`ddos`	both	$0.928 \to 0.124$	$+0.931$	$0.080 \to 0.865$ (0.821)
`syn_ddos`	size	$1.000 \to 0.144$	$+0.932$	$0.879 \to 1.000$ (1.000)
`syn_ddos`	both	$0.973 \to 0.129$	$+0.923$	$0.973 \to 0.999$ (0.998)
`portscan`	both	$0.972 \to 0.105$	$+0.835$	$0.011 \to 0.679$ (0.999)
`portscan`	size	$1.000 \to 0.104$	$+0.835$	$0.013 \to 0.479$ (0.999)
`ssdp_flood`	iat	$0.110 \to 0.081$	$+0.535$	$0.001 \to 1.000$ (1.000)
`ssdp_flood`	both	$0.965 \to 0.003$	$+0.374$	$1.000 \to 1.000$ (1.000)
No footprint to undo				
`ldap_ddos`	both, iat	$\leq 0.18 \to \approx 0$	≈ 0	$1.000 \to 1.000$ (1.000)
`ldap_ddos`	size	—	$+0.000$	$1.000 \to 1.000$ (1.000)
`ssdp_ddos`	both, iat	$\approx 0 \to \approx 0$	≈ 0	$0.998 \to 0.998$ (0.998)
`ntp_ddos`	both, iat	$\approx 0 \to \approx 0$	≈ 0	$0.983 \to 0.986$ (0.998)
Footprint reduced, but one judge is uninformative				
`dns_ddos`	size	$1.000 \to 0.302$	-0.002	$0.986 \to 0.765$ (0.986)
`ssdp_flood`	size	$1.000 \to 0.003$	$+0.549$	—
Purification harms an attack that was not evading				
`dns_ddos`	iat	$\approx 0 \to \approx 0$	-0.006	$0.993 \to 0.756$ (0.998)
`dns_ddos`	both	$\approx 0 \to \approx 0$	-0.008	$0.986 \to 0.762$ (0.996)
`portscan`	iat	$\approx 0 \to 0.007$	$+0.000$	$0.994 \to 0.811$ (0.999)
`brute_force`	size	$1.000 \to 0.188$	-0.002	$0.993 \to 0.963$ (0.994)
`mirai`	size	$1.569 \to 0.973$	$+0.271$	$0.257 \to 0.005$ (0.008)

probability to begin with: the gain metric is bounded by the detection gap, so on detector-blind sources purification can seem invisible to it. On `ssdp_flood size` the other column is the one that fails—the footprint is all but erased ($1.000 \to 0.003$, gain $+0.549$), but the independent NIDS cannot be scored on that stream at all, because none of its packets survive L7 realization.

The fourth regime is the one the earlier framing missed, and it only becomes visible once an independent judge is used. On sources where TANTGen never succeeded in evading, purification actively harms detection. Through the independent NIDS, `dns_ddos` falls from 0.986 to 0.762 on `both` and from 0.993 to 0.756 on `iat`; `portscan_iat` falls from 0.994 to 0.811; `brute_force_size` from 0.993 to 0.963. The denoiser is doing what it was trained to do, moving the flow toward the clean manifold, but on these flows the attack's fingerprint and the clean manifold were never far apart, so what it removes is signal the protected NIDS was still using.

Two realization caveats must be stated. First, a size edit on an application-layer text protocol (the SSDP/HTTP `M-SEARCH` messages in `ssdp_flood both/size`) corrupts the L7 grammar even when the IP and UDP lengths and checksums remain valid, and `tshark` flags these as malformed. Rather than count them as compliance failures, since they are artefacts of byte-overlay realization on a text protocol and not of the purifier, the $450{,}758$ such packets are reported separately as unrealizable at L7, and the



Figure 5.3: Purification gain against perturbation footprint, one mark per source–mode pair (27 of 30; the three pairs whose pairing records no perturbed-dimension MSE are omitted). Top right: the footprint is erased and the detection gap closed. Bottom left: TANTGen barely moved the features, so purification is correctly inert. Bottom right: the `size` variants of the reflection floods carry a full footprint, but the TCN is near chance on them, so no gain is measurable through this judge. The three `mirai` pairs sit at footprints of 1.6–2.5 and are shown in the broken-off panel on the right, past the axis break. Note that the axis is the in-distribution gain, so the sources on which purification harms independent-NIDS detection are not distinguishable here; Table 5.5 reports that separately.

97.22% rate is computed over the $2{,}412{,}357$ realizable ones. For `ssdp_flood` `size` that exclusion is total: every packet of that stream is L7-unrealizable, so it has no compliance or independent-NIDS figure at all. Second, the total compliance rate is pulled down by just two sources: `ddos` at 0.84 ($61{,}447$ of $391{,}270$ realized packets flagged) and `syn_ddos` `iat` at 0.178, the latter a thin slice, since only 927 of that source's $10{,}748$ test pairs survive the equal-length pairing guard, and one whose $2{,}088$ packets fail on bad TCP checksums after the rewrite rather than on malformation, which points at the realization step rather than at the reconstructor. Every other source is at or above 0.99. Neither is a claim about purification quality, but they do bound how far the traffic-space results can be pushed.

To make these aggregates concrete, it is worth following a single real packet through the pipeline. Figure 5.4 traces one flow drawn from the held-out test split, a four-packet Mirai SYN-flood burst from `192.168.2.108:35220` to `52.25.66.250:8280` under the `size` variant. In the original capture each datagram is 44 bytes, a 20-byte IPv4 header followed by a 24-byte TCP header with no application payload. TANTGen appends 29 bytes of payload to every packet, inflating the IPv4 total-length field to 73 bytes; this length shift is the whole perturbation the `size` variant applies, and it is what pushes the flow off the clean manifold. The reconstructor recovers the original length, and the traffic-space realization of Section 4.5.3 trims the appended bytes back off, returning every datagram to 44 bytes, a residual of exactly zero on the manipulated field. All twelve packets across the three stages pass `tshark` dissection with

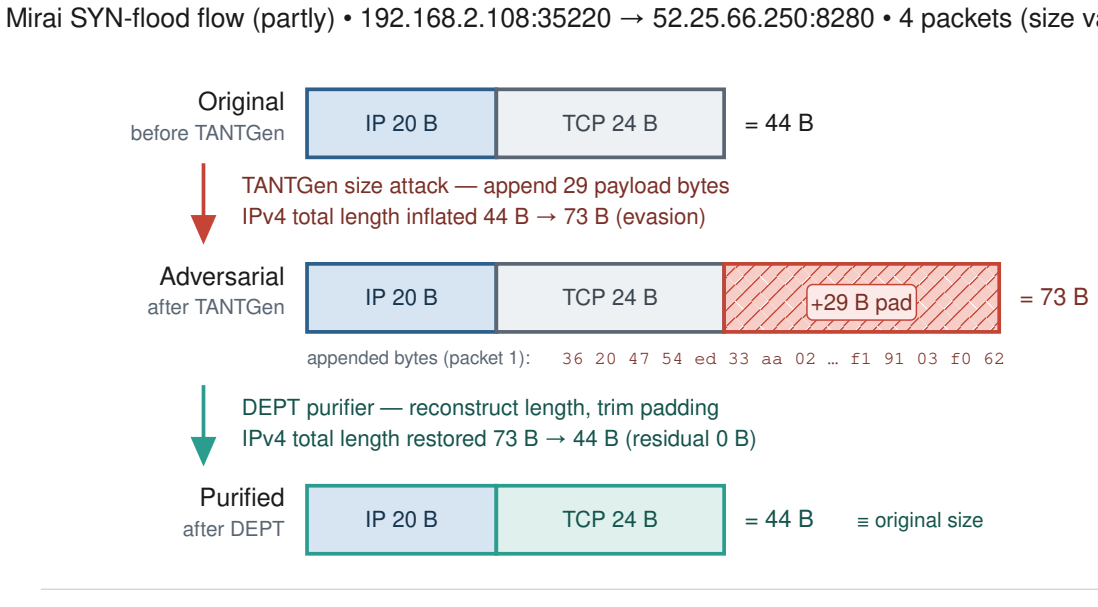


Figure 5.4: A single real packet through the purification pipeline, drawn from the held-out test split. Under the `size` variant, TANTGen appends 29 payload bytes to each Mirai SYN packet, inflating the IPv4 total length from 44 to 73 bytes; the reconstructor restores the length and the realization trims the padding, returning the datagram to 44 bytes with zero residual on the manipulated field. Every stage passes `tshark` dissection. Byte counts and the appended bytes are read directly from the three captures with `packet_example/extract_packet_example.sh`.

no malformed frames and no bad checksums, and because the `size` variant perturbs length alone the inter-arrival times are identical between the clean and adversarial captures, so packet size is the only quantity that changes. The $17.0\times$ reduction in perturbed-dimension MSE and the restored detection reported above are, at the granularity of one packet, this $44 \rightarrow 73 \rightarrow 44$ restoration.

5.7 Ensemble Robustness under a Gradient-guided Attacker (RQ1)

Every result so far was produced against TANTGen, and TANTGen cannot answer RQ1. It is black-box and gradient-free, so it leverages no gradient path; measuring it tells us the ensemble detects its perturbations, not what the ensemble's heterogeneity is worth against an adversary that differentiates. This section supplies that adversary.

5.7.1 The Attacker

The attacker is given white-box access to the TCN (architecture, weights and gradients) and nothing for the Random Forest, whose non-differentiability means there is no gradient to target. It crafts one perturbation against the TCN, using PGD [36], and that same perturbation is then replayed, unchanged,

through each target's own featurisation and scored. This strategy is what makes the comparison valid: were the attacker allowed to re-optimize per target, it would no longer be measuring transfer.

The perturbation is increase-only padding and delay, per packet, applied on top of TANTGen's own, so the cost reported is the extra budget an attacker must spend. The padding bytes are drawn at random (`attack.pad_byte_mode`); this is a meaningful detail, because zero and random padding move the Random Forest's payload-entropy features, its largest feature group, in opposite directions, while the TCN's gradient sees no payload bytes and so cannot distinguish the two. Budgets are denominated in bytes and seconds rather than in an ℓ_∞ ball: the IPAT channel is standardized with a standard deviation of $211{,}047$ seconds while the 1088 nPrint channels are bits in $\{-1, 0, 1\}$, so a single $\epsilon = 0.01$ would buy the attacker ± 2110 seconds of delay and nothing whatsoever in the bit channels. The feasible set (increase-only, integer bytes, capped at the Maximum Transmission Unit (MTU), inert on non-IPv4 packets) is enforced as a projection itself.

Three controls guard the result, because a defense evaluated with a weak attack cannot be fully trusted [56, 57]. First, a query-matched random search: the same feasible set and budget, an equal number of forward passes, and no gradient. Second, AutoPGD [54] via the ART library [55], a parameter-free optimizer, as an independent check that the search is not simply under-tuned; it agrees, finding the same 100% evasion of the TCN at the same budget. Third, the reported attacker is the best of PGD and random search per flow, so no result rests on a favourite optimizer.

5.7.2 How the Numbers Should Be Read

Two conventions are critical, and both work against the defense rather than for it.

ASR is measured only over the flows a target detects unperturbed. This is the standard robust-accuracy convention, and here it is the difference between a measurement and an artifact: of the $9{,}000$ sampled flows, the TCN detects only $3{,}374$ at its matched threshold, against $7{,}510$ for the forest and $7{,}597$ for the stack, so an unconditional ASR would mostly be counting flows the detector never caught. A target that does not detect a flow in the first place cannot be evaded on it.

Thresholds are set at a matched false-positive rate on unperturbed benign traffic, never at a raw 0.5. The three targets are calibrated very differently, with median verdicts on benign traffic of 0.043 for the forest, 0.379 for the TCN, and 0.002 for the stack, so a fixed threshold would reward whichever model happens to say "adversarial" least often. At a common 5% false-positive rate the thresholds are 0.4535, 0.9820, and 0.3658 respectively.

Finally, the curves are reported as a monotone envelope over the budget grid. The feasible sets are nested, so an attacker permitted 1460 bytes may simply spend 128; a rational attacker's success cannot fall as its budget grows, and any measured dip is the optimizer faltering rather than the defense strengthening.

5.7.3 Results

Table 5.6: Attack success rate at a matched 5% false-positive rate, for one perturbation crafted against the TCN and scored on all three targets. The budget is the attacker's per-packet cap, so the per-flow spend, summed over the packets of a flow, legitimately exceeds it; both spend columns are medians of what the attacker actually paid. Denominator: flows the target detects at $\delta = 0$, which is $7{,}510$ for the RF, $3{,}374$ for the TCN, and $7{,}597$ for the stack, of $9{,}000$ sampled. Lowest attack success in each row in **bold**.

Budget (B/pkt)	**Spend** (B/pkt)	**Delay** (ms/pkt)	**Spend** (B/flow)	**RF**	**TCN**	**Stack**
$\delta = 0$	0	0	0	0.0%	0.0%	0.0%
8	3.0	0.004	13	8.6%	40.5%	**1.7%**
32	14.0	0.041	47	10.7%	50.1%	**4.7%**
128	49.5	0.419	167	13.1%	52.2%	**8.5%**
512	159.8	3.677	537	**18.4%**	53.3%	18.9%
1460	402.0	28.513	1,574	36.0%	54.5%	**25.5%**
median budget to evade				b512	b8	b512
never evaded, any budget				64.0%	45.5%	**74.5%**

A – The stack costs the attacker more than either base learner alone, at every budget an attacker would plausibly pay. This is RQ1's answer. At the three smallest non-zero budgets the fused stack concedes 1.7%, 4.7%, and 8.5% attack success against the forest's 8.6%, 10.7%, and 13.1% and the TCN's 40.5%, 50.1%, and 52.2%; across the whole grid 74.5% of flows never evade the stack at any budget, against 64.0% for the forest and 45.5% for the TCN, whose median evading flow needs only the smallest budget on the grid. The perturbation that breaks the TCN it was crafted against transfers poorly to the forest, and the fusion of the two is harder than either. For scale, TANTGen's own edits average about 29 to 32 padding bytes per packet on the sources where it perturbs most, which places a realistic attacker in the 32 to 128 byte region of the grid, where the stack concedes at most 8.5%.

One qualification belongs here as well. The ordering is not uniform across the whole grid: at 512 bytes the stack (18.9%) and the forest (18.4%) are level, and the stack pulls ahead again only at 1460 bytes (25.5% against 36.0%). The claim is therefore that the stack strictly dominates the TCN everywhere and the forest everywhere except one grid point, where they are roughly equal.

B – The meta-learner shifts its reliance onto the Random Forest as the TCN is degraded. This is the design claim of Section 4.4.3 stated as a prediction, and it survives the test. Ablating the meta-learner's two base-learner token slots, and measuring how far its verdict moves when each is handed back its clean value, the share attributable to the forest goes from 0.000 at $\delta = 0$, where there is nothing to compensate for, to 0.529 as soon as the attacker spends anything, rising to 0.563 at 128 bytes and staying between 0.54 and 0.56 thereafter, with between $5{,}869$ and $6{,}458$ of the $9{,}000$ flows registering a moved verdict. The stack does lean on the learner the attacker cannot differentiate. The

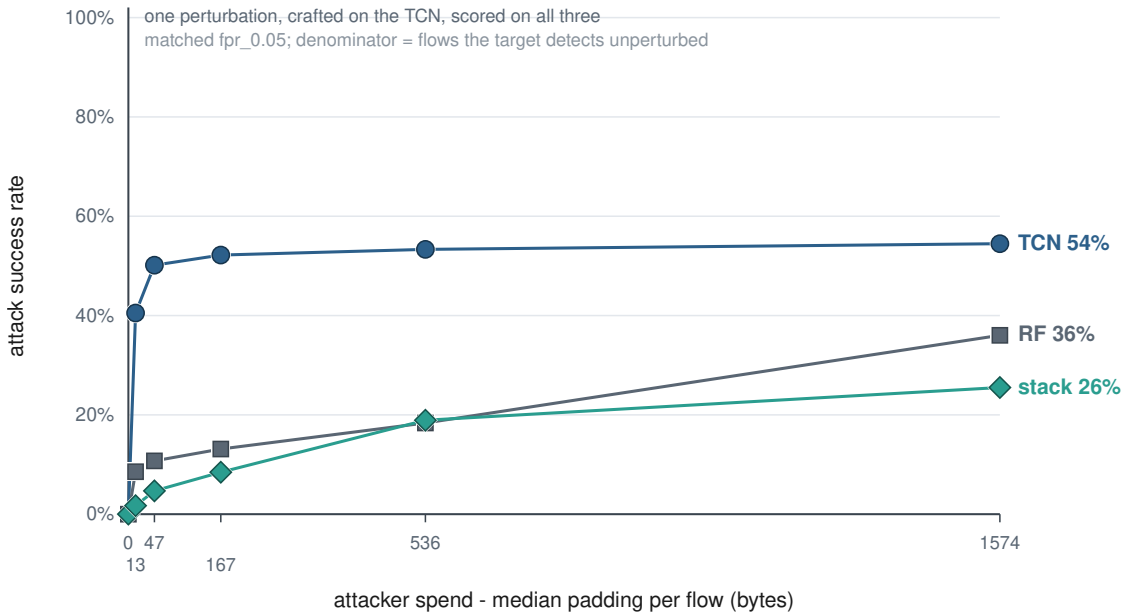


Figure 5.5: Attack success against each target as the attacker's spend grows, for a single perturbation crafted against the TCN with white-box gradients and replayed unchanged through the other two. The TCN is the attacker's own target; the Random Forest is reachable only by transfer, since it exposes no gradient. Success is measured over the flows each target detects unperturbed, at a matched 5% false-positive rate.

behaviour is visible even before any attack: where the TCN is blind at $\delta = 0$ and the forest is not, as on `ldap_ddos`, `ntp_ddos both/iat`, `portscan_iat` and `syn_ddos_iat`, the stack inherits the forest's detection of ≈ 1.00. It also inherits the forest's failures, reading ≈ 0.00 wherever the forest does, as on `ssdp_ddos_size` and `ntp_ddos_size`, and it is not uniformly the harder target per source: on `ntp_ddos_both` and `ssdp_ddos_both` the stack is easier to evade at 128 bytes than the forest alone (20% and 43% against 1%).

C – Gradient guidance gives this attacker no aggregate advantage. Query-matched random search does not merely match PGD on the TCN, it beats it at every budget, by 40.1% against 37.2% at 8 bytes and 52.8% against 50.7% at 1460. The cause is presumably representational: nPrint encodes packet size as 16 binary total-length columns, so $255 \to 256$ flips nine of them at once and neighbouring byte counts sit far apart in the model's input space. The loss surface over packet size is correspondingly jagged, and a descent direction computed there carries little information.

The comparison is aggregate, not universal: since the better variant is selected per flow, the best-of curve exceeds both (40.5% at 8 bytes, 54.5% at 1460), so PGD still carries flows random search misses. ART's AutoPGD confirms the search is not simply under-tuned: on the relaxed problem all three attackers push every one of the sampled flows past the decision boundary, with median margins of -0.79 to -0.85. This is a property of the representation the defense happens to use, not a robustness

claim, and it bounds the RQ1 result in a specific way: what has been demonstrated is that adding a non-differentiable component raises evasion cost against a strong search-based attacker, not that gradient-based attacks are weak in general.

5.8 End-to-End System Evaluation

5.8.1 Detect-then-Purify, End to End

Composing the stages, the end-to-end effect of DEPT can be measured either through the protected detector's own verdicts or through an independent one. The internal view is reported first, and briefly, because it is the weaker of the two: through DEPT's own TCN, the mean $P(\text{adv})$ on a purified flow falls from 0.675 to 0.428 against a clean-flow level of 0.421, so purification returns flagged flows to the benign space as our own detector sees it. On the structurally perturbed sources this is the difference between a near-certain detection ($P(\text{adv}) \approx 0.99$ on `portscan` and `ddos`) and a near-certain pass (≈ 0.06–0.15).

These figures are measured through DEPT's own TCN standing in for the protected NIDS, which makes them an internal view: they show that purification returns a flow to the manifold as scored by the same detector family the loop was trained around. The credible end-to-end question is different, and this thesis answers it directly rather than deferring it: what is TANTGen's attack-success rate against the independent NIDS each attack was actually built to evade, before and after DEPT is placed in front of it?

Table 5.7 reports that deployment baseline. For every attack, three realized full streams are scored through its true target NIDS: **Kitsune** (KitNET + AfterImage) for the two Kitsune-native attacks, and TANTGen's own **ML-NIDS** (the per-attack CICFlowMeter flow classifiers) for the eight CIC-IDS2017 and CIC-DDoS-2019 attacks, reusing in each case the very model TANTGen calibrated its evasion against, so that no DEPT component is ever asked to grade itself (again). The clean stream carries the original-attack size and timing and states the theoretical ceiling (the NIDS working as intended); the adversarial stream carries TANTGen's perturbation and in principle falls to the evasion floor; and the DEPT-defended stream is produced by the full gated pipeline of Section 4.2, in which the meta-learner flags each flow and only the flagged flows are purified while the rest pass through unchanged, so that detection should climb back from the floor toward the ceiling. The three streams are realized onto the same adversarial packets, differing only in the size and timing the purifier touches (Section 4.5.3), which makes the comparison a controlled one. The headline metrics are the attack-success rate (one minus the NIDS detection rate) before and after DEPT, and the detection-restoration ratio, the fraction of the adversarial-to-clean detection gap the pipeline recovers.

Aggregated over $2{,}704{,}816$ scored flows, DEPT cuts the protected NIDS's attack success from 21.1% to 11.6%, a reduction of 9.5 points, and recovers 49.3% of the detection TANTGen had destroyed. That is the headline, but read alone it may overestimate the system, because the average is taken over thirty

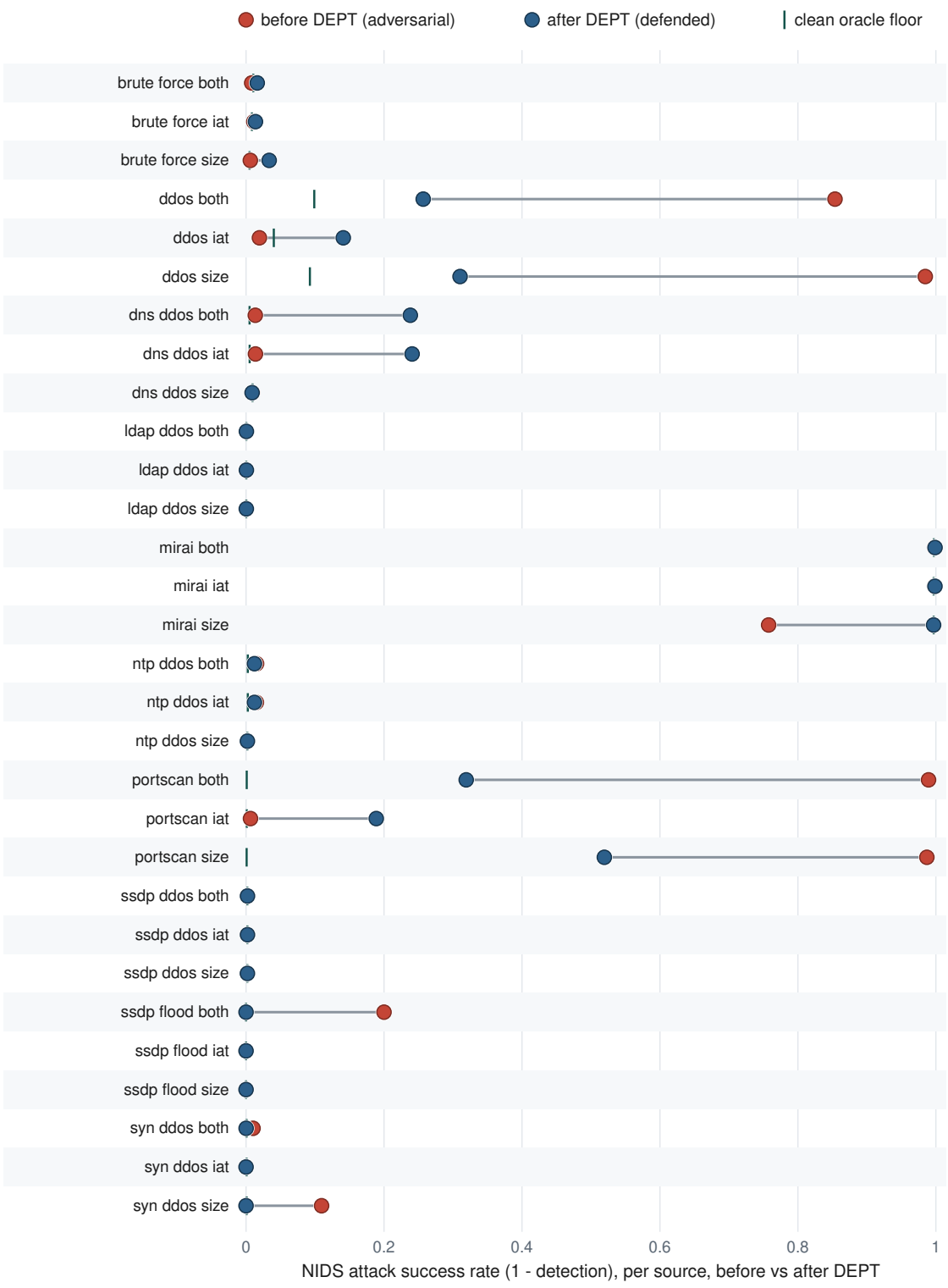


Figure 5.6: The protected NIDS's attack-success rate on each source, before DEPT (on TANTGen's adversarial traffic) and after DEPT (through the gated detect-then-purify pipeline), with the clean-oracle floor marked; the connecting line is the attack success the pipeline removes (or adds). Each source is scored through the NIDS it was built to evade.

Table 5.7: Deployment baseline. The protected NIDS's attack-success rate (ASR, one minus its detection rate) on the clean original attack (oracle floor), on TANTGen's adversarial traffic (before DEPT), and on the gated detect-then-purify pipeline's output (after DEPT), each attack scored through the NIDS it was built to evade and aggregated over its three perturbation modes by flow count. Δ is the ASR the pipeline removes, positive being an improvement. Two rows are degenerate and marked ‡: on `mirai` Kitsune detects almost nothing even on the clean original attack, and on `ssdp_flood` it detects everything in all three regimes, so neither can show restoration.

Attack	NIDS	ASR clean	ASR before	ASR after	Δ
ddos	ML-NIDS	0.078	0.616	0.234	+0.382
portscan	ML-NIDS	0.001	0.661	0.342	+0.319
syn_ddos	ML-NIDS	0.001	0.047	0.000	+0.047
ntp_ddos	ML-NIDS	0.002	0.011	0.009	+0.002
ldap_ddos	ML-NIDS	0.000	0.000	0.000	+0.000
ssdp_ddos	ML-NIDS	0.002	0.002	0.002	−0.000
brute_force	ML-NIDS	0.008	0.009	0.021	−0.012
dns_ddos	ML-NIDS	0.006	0.013	0.192	−0.179
mirai‡	Kitsune	0.997	0.918	0.998	−0.080
ssdp_flood‡	Kitsune	0.000	0.000	0.000	+0.000
Overall		**0.019**	**0.211**	**0.116**	**+0.095**

source–mode combinations whose individual outcomes differ significantly.

Figure 5.6 makes the structure plain, and it is worth targeting in three parts. The gain is real and large where TANTGen actually evaded. On `ddos` the pipeline removes 38 points of attack success and on `portscan` 32, and at mode granularity the strongest cases are `ddos_size` (+67.4 points), `portscan_both` (+67.0), `ddos_both` (+59.7), and `portscan_size` (+46.8). These are exactly the structural `both`/`size` edits that the per-source purification results predicted would be recoverable. Most combinations are untouched because there was nothing to recover: on `ldap_ddos`, `ntp_ddos`, `ssdp_ddos` and `brute_force`, TANTGen never evaded in the first place (adversarial detection is already 0.98 to 1.00), so the pipeline moves the number by less than a point in either direction. And on a minority of combinations the defense makes matters worse. `dns_ddos` loses 22.5 points on `both` and 22.7 on `iat`, `portscan_iat` loses 18.2, `ddos_iat` loses 12.2, and `mirai_size` loses 23.9. The mechanism is the one Section 5.6 identified: the gate flags essentially all flood traffic, so flows the NIDS was still catching get purified, and purification removes signals it was using.

The gate's behaviour is a concerning cause and is worth quantifying. Its flag rate is close to 100% on every flood source and close to zero on several `size` variants (0.0% on `ssdp_ddos`, 0.2% on `ntp_ddos`, 0.8% on `ldap_ddos`, 0.9% on `dns_ddos`), which is the same reflection-flood `size` blind spot the base learners showed in Table 5.2, now visible in the deployed pipeline. One consequence is that the gated pipeline and an ungated one that purifies everything are nearly indistinguishable here: the `purified_all` upper bound reaches 0.8831 detection against the gated 0.8836. The gate as built is therefore not doing much work, and the finding that matters is not that the gate is imperfect but that it is answering the wrong

question. It predicts whether a flow is adversarial; what this pipeline specifically needs is a predictor of whether purifying this flow will help, which we have seen is a different story.

Split by detector family, the two halves diverge: the eight ML-NIDS sources gain 9.9 points of ASR reduction over $2{,}622{,}023$ flows, while the two Kitsune sources lose 1.7 points over $82{,}793$. The Kitsune figure rests almost entirely on `mirai_size`, where TANTGen's size edit had inadvertently raised Kitsune's detection from 0.003 to 0.242 and DEPT removed that self-sabotage, returning it to 0.003. This is the purifier working correctly and the defense losing by it, which is certainly an instructive combination to say the least.

Two methodological notes come with these numbers. First, because the defended stream runs DEPT's detector and purifier over all of an attack's flows for full-volume realism, some of those flows were seen in training; restricted to the held-out test flows the defended detection rate is 0.9061 over $372{,}321$ flows, slightly above the full-stream 0.8836, so the deployment view is not flattered by the training overlap. Second, Kitsune is a partly volumetric detector, so its detection rate is only meaningful on full streams such as these, which is exactly why the baseline feeds full realized traffic rather than a flow subset; the reconstructor evaluation of Section 5.6, which by design sees only the held-out test pairs, therefore leads with the restoration ratio instead, while the flow-level ML-NIDS is unaffected either way.

5.8.2 One Global Model or Ten Specialists?

Every result so far concerns a single set of models trained across all ten sources at once. That was a design decision assumed at the beginning, and it invites the obvious question: would ten specialists, as per usual in the literature, each trained on one attack's own benign and adversarial flows, do better? The four model stages were therefore re-run once per attack, giving ten independent RF/TCN/meta-learner/reconstructor sets, and Table 5.8 compares each specialist meta-learner against the global one on the same slice and the same axis.

The re-run varies the training scope and holds everything else fixed. Each specialist sees only its own source's benign capture and that source's three adversarial variants, but inherits the architectures, the grid searches, and the flow-atomic, seed-42 $70/15/15$ split protocol of Section 5.2.1, so each is scored on exactly the flows the global model is scored on once its test set is restricted to that slice. The variable under test is only what a model may contrast a manipulated flow against, one source's benign pool or ten.

Neither regime wins outright, and the variance across sources is larger than any average difference between them. On 13 of the 30 combinations the two are indistinguishable, both already at or near 1.000. Where they differ the pattern is systematic. Specialists win on the `both` and `iat` modes of the reflection floods and on timing-only edits generally: `ntp_ddos` gains 0.346 and 0.347, `syn_ddos_iat` gains 0.194, `ssdp_ddos` gains 0.073 on both modes, and `portscan_iat` gains 0.066. Training on one attack lets

Table 5.8: Meta-learner AUROC@MAX against same-source benign flows, for a model trained on all ten sources (global) and for one trained on that attack alone (spec.). Δ is specialist minus global, positive favouring the specialist. Differences below 0.005 are shown as ≈ 0.

	both			iat			size		
Source	spec.	global	Δ	spec.	global	Δ	spec.	global	Δ
`brute_force`	0.995	0.981	$+0.014$	0.995	0.979	$+0.016$	0.997	0.989	$+0.007$
`ddos`	1.000	1.000	≈ 0	1.000	0.967	$+0.033$	1.000	1.000	≈ 0
`dns_ddos`	1.000	1.000	≈ 0	1.000	1.000	≈ 0	0.192	0.443	-0.251
`ldap_ddos`	1.000	1.000	≈ 0	1.000	1.000	≈ 0	0.435	0.476	-0.041
`mirai`	0.689	0.724	-0.035	0.687	0.719	-0.033	0.740	0.734	$+0.006$
`ntp_ddos`	1.000	0.654	$+0.346$	1.000	0.653	$+0.347$	0.313	0.493	-0.180
`portscan`	0.998	0.998	≈ 0	0.824	0.757	$+0.066$	0.996	0.997	≈ 0
`ssdp_ddos`	1.000	0.927	$+0.073$	1.000	0.927	$+0.073$	0.355	0.497	-0.142
`ssdp_flood`	1.000	1.000	≈ 0	1.000	1.000	≈ 0	1.000	1.000	≈ 0
`syn_ddos`	1.000	1.000	≈ 0	1.000	0.806	$+0.194$	1.000	1.000	≈ 0

the model key on that source's own timing signature instead of averaging over nine unrelated benign pools. The global model wins on the reflection floods' `size` variants, but this is a hollow victory on the slices where both regimes are already below chance: it is less confidently wrong (0.443 against 0.192 on `dns_ddos`), not right.

Two findings from this comparison bear on the ensemble design rather than on the deployment choice. First, the stacking principle of Section 5.5 reproduces itself for each specialist rather than being an artifact of training across sources. `mirai` is the one source where the global model is genuinely better ($+0.035$, $+0.033$), and it is also where the specialist stack fails its own base learner most profoundly: the per-attack meta-learner reaches 0.705 overall while its own Random Forest reaches 0.889. `portscan` shows the same sign more mildly (0.939 against 0.999), and on the remaining eight sources the specialist stacker matches or beats its own forest, so narrowing the training scope neither creates the pathology nor cures it. Second, the per-attack TCNs collapse entirely on the reflection floods, scoring exactly 0.500 on `ldap_ddos` and `ssdp_ddos` and 0.507 to 0.531 on the others, so those specialist stacks are riding on their forest alone. On the sources whose flows are long enough to convolve over, the same specialists are untroubled by the narrower pool (0.990 on `ddos`, 1.000 on `ssdp_flood`, 0.980 on `brute_force`), which places the collapse on flow length rather than with training scope, exactly where Section 5.4 placed the global TCN's version of it. Both observations point the same way as Section 5.5: the fusion layer needs cross-source diversity to learn when to trust which stream, and does not particularly thrive without it.

The practical conclusion is that a single global model is the right default for this corpus specifically. The accuracy case for specialists is confined to a handful of slices, and poses a predictable deployment cost: a specialist regime needs a router that names the attack before a model can be chosen, which is a version of the very classification problem the detector exists to solve. Specialists stay worth the trouble in the narrow case the table identifies, a known high-rate flood whose evasion is expressed in timing, where `ntp_ddos` alone gains 0.346 and 0.347. The reflection-flood `size` blind spot, by contrast, appears

in both regimes, which localizes it in the feature representation rather than in the training scope.

5.9 Discussion and Limitations

A – The research questions. RQ1 is answered affirmatively: an adversary handed the TCN's gradients, and nothing for the forest because a forest has nothing to give, pays more to evade the fused stack than either component at every plausible budget, and 74.5% of flows never evade it at all against 45.5% for the TCN (Section 5.7, bounded by Section 5.9.1). RQ2 is answered affirmatively but conditionally: through the independent NIDS each attack was built to evade, purification recovers 63.0% of the destroyed detection, 99.9% of it where the evasion had actually succeeded, while emitting 97.2% compliant packets. The condition is that the attack left a footprint the protected detector could see; where it did not, purification ranges from inert to actively harmful. Wall-clock cost is not measured here (Chapter 6).

B – Purification without a good gate can subtract as well as add. The evaluation's most consequential negative finding, and one visible only under an independent detector: where TANTGen never actually evaded, purifying lowers the protected NIDS's detection, by 22.5 and 22.7 points on `dns_ddos both/iat`, 18.2 on `portscan_iat` and 12.2 on `ddos_iat`, because there the attack's fingerprint and the clean manifold were never far apart, so what the denoiser correctly removes is signal the NIDS was presumably using. The gate fails to prevent this, flagging nearly all flood traffic, so gated and ungated pipelines reach 0.8836 and 0.8831 detection: it predicts whether a flow is adversarial, where the pipeline needs a predictor of whether purifying this flow will help.

C – The reflection floods' `size` variants fail below chance. On `dns_ddos`, `ldap_ddos`, `ntp_ddos` and `ssdp_ddos` they score 0.213–0.476 for the forest and 0.443–0.497 for the meta-learner against same-source benign flows (Table 5.2), so such a flow resembles generic benign traffic more than that flood's own unedited flows do. The cause is the feature representation rather than capacity, tuning or training scope: the per-attack specialists fail on the same slices (Table 5.8), the purifier cuts the footprint sharply ($1.000 \rightarrow 0.302$ on `dns_ddos_size`) without moving any verdict, and the deployed gate flags under 1% of these flows. DEPT's 24 prefix statistics and 1088 header bits cannot express whatever separates a padded single-response UDP flood from an unpadded one.

D – A stacker beaten by its own base learner. The meta-learner's 0.9334 AUROC sits below the forest's 0.9813 at every horizon from packet 2 onward (Table 5.3), widening per source to 0.653 against 0.999 on `ntp_ddos` and 0.927 against 1.000 on `ssdp_ddos`; the cause is likely Section 5.5's under-weighted forest logit, and the specialist stacks reproduce it (Section 5.8.2). Fusion still buys the only packet-1

verdict available, worth nearly ten AUROC points over the TCN alone, and the evasion robustness of Section 5.7, but a fusion able to fall back to its best base learner is the clearest improvement available here.

5.9.1 Scope of the Gradient-ablation Result (RQ1)

Four boundaries limit what Section 5.7 accomplishes.

- ASR is measured over the flows a target detects unperturbed, and at $\delta = 0$ the TCN detects between nothing and 2% of flows on seven of the ten sources, leaving its ASR undefined there; four sources have a median of two packets against a receptive field of 29. That arm of the comparison rests on a minority of the corpus.
- Some flows are unreachable at any budget. On the `both` and `size` modes of `ddos` and `portscan`, and on `ssdp_flood_size`, both the forest and the stack hold the attacker to 0% through 1460 bytes per packet, which says only that padding and delay alone cannot overcome that much model confidence.
- The timing channel has a representational floor: a delay below $\approx 3.07\,\mu$s, one float32 ULP of the standardized IPAT channel, never reaches the tensor, and `dns_ddos`'s median inter-arrival time ($2.38\,\mu$s) sits under it.
- What was measured is a search-based attacker with a gradient available to it, since query-matched random search beats PGD at every budget. A non-differentiable component raises evasion cost; gradient attacks are not therefore weak in general.

So the result stands as the following: one perturbation crafted against the TCN costs the attacker materially more against the stack than against either component, and the meta-learner's reliance migrates toward the learner the attacker cannot differentiate.

6

Conclusion

Contents

6.1 Summary of Contributions 67
6.2 Answers to Research Questions 68
6.3 System Limitations 70
6.4 Future Work 71

This thesis set out to defend a learning-based NIDS against a black-box attacker that reshapes live attack traffic into protocol-compliant flows the detector labels benign. Its answer is DEPT, a streaming defense that detects the manipulation mid-flow and undoes it before the protected system consumes it. This chapter summarizes what was delivered (Section 6.1), answers the research questions (Section 6.2), states the limitations (Section 6.3), and lays out where the work continues (Section 6.4).

6.1 Summary of Contributions

Five contributions were announced in Section 1.4; each can now be restated against the chapters that substantiated them.

The first is the packet-to-tensor pipeline of Section 4.3, which turns raw captures into a masked, bit-level representation of each flow, identity fields removed so that the models learn behaviour rather than addresses, at two granularities: a dense per-packet sequence for the TCN and sparse aggregated checkpoints for the RF. Every model is trained and evaluated on the same flow-atomic split of the same tensors, so the comparisons of Chapter 5 are like-for-like by construction.

The second is the heterogeneous detection ensemble of Section 4.4, an RF and a TCN from deliberately different architectural families fused at (almost) every packet by a small attention-based meta-learner, together with the streaming axis that scores every model by how early, and not only how well, it separates manipulated from clean flows. That axis did not show what the design had predicted, and the evaluation reports it as measured: the Random Forest alone ranks flows best from packet 2 onward, and the fusion's advantages are narrower, the packet 1 verdict and a higher cost to a gradient-guided attacker (Table 5.3, Table 5.6).

The third is the purification stage of Section 4.5: a causal Transformer that predicts a residual correction over a flagged flow, linked afterwards to a realization step that writes the corrected features back into packets and validates them with a dissector (Table 5.4). Realization closes the inference path, so the protected NIDS receives traffic rather than tensors, at the cost of being scoped to the two axes this threat model perturbs. The defense therefore does not stop at feature space, as the problem statement required from the start.

The fourth is the deployment baseline of Section 5.8.1, which measures DEPT as what it is, a defense in front of a protected NIDS: the gated pipeline is run over clean, TANTGen-perturbed, and DEPT-defended traffic, each stream scored through the real NIDS the attack was built to evade, so that the end-to-end claim rests on an independent judge rather than on DEPT grading it itself.

The fifth is the evaluation corpus and its insightful accounting: ten benign sources from three public datasets, each manipulated by TANTGen in three perturbation modes and evaluated per source and per mode rather than only in aggregate (Chapter 5). The corpus is reusable, and the per-source analysis is as much a contribution as the headline numbers, because the aggregate hides two very different populations: sources on which the defense is essentially solved and sources on which no model rises much above chance.

6.2 Answers to Research Questions

Section 5.1 fixed two research questions, one per pillar of the system; both can now be answered, with the honesty we aimed for.

RQ1 (ensemble robustness) asked whether heterogeneous stacking measurably raises the cost of a transferable, gradient-guided evasion attack. It measurably does. Because TANTGen is gradient-free

and cannot measure a gradient path, the evaluation employed a stronger adversary: white-box access to the TCN, nothing for the RF since a forest has no gradient to give, and a single perturbation spent against all three targets at once (Section 5.7). That perturbation transfers poorly from the TCN to the forest, and the fused stack is costlier still: at the three smallest budgets it concedes 1.7%, 4.7%, and 8.5% attack success against the forest's 8.6%, 10.7%, and 13.1% and the TCN's 40.5%, 50.1%, and 52.2%, and 74.5% of flows never evade it at any budget against 45.5% for the TCN alone. The design's prediction that the meta-learner would lean on the RF as the TCN degrades survived: the forest's share of the stack's verdict goes from zero at the beginning to 0.53 as soon as the attacker spends anything, and stays there.

Three qualifications bound the claim. The ordering is not uniform: stack and forest are level at 512 bytes before the stack pulls ahead again; the question is only well-posed where the base learners detect the flow at all, which excludes several sources, and elsewhere confidence is too large for padding and delay to overcome at any budget (Section 5.9.1); and, against RQ1's own phrasing, the gradient bought this attacker nothing, since query-matched random search beats PGD at every budget because nPrint's binary total-length encoding makes the loss surface over packet size jagged. What was demonstrated is therefore that a non-differentiable component raises evasion cost against a strong search-based attacker, a finding highly attributed to representation as much as the ensemble architecture (Section 6.4).

RQ2 (purification effectiveness) asked whether the reconstructor moves a flagged flow back onto the benign manifold while emitting a realizable packet sequence. It does, under a condition the evaluation made explicit, and the answer does not rest on DEPT grading alone. Scored through the independent NIDS each attack was built to evade, purification recovers 63.0% of the destroyed detection overall and 99.9% on the Kitsune-native sources where TANTGen's evasion had actually succeeded, while reconstruction error on the perturbed dimensions falls by $17.0\times$ and the realized packets pass `tshark` 97.2% of the time (Section 5.6). End to end, the gated pipeline cuts the protected NIDS's attack-success rate from 21.1% to 11.6% (Section 5.8.1).

The condition is, however, specific. Where TANTGen barely moved the features the purifier is correctly unmoved; where it moved them without evading, purifying actually lowers the protected detector's detection, by as much as 22 points on `dns_ddos` and 18 on `portscan_iat`, because what the denoiser removes is signal the NIDS was still using. The aggregate gain is therefore a net of large recoveries on six source–mode combinations against real regressions on five others, and the gate meant to prevent the latter does not, since it predicts whether a flow is adversarial rather than whether purifying it will help. RQ2 is answered affirmatively only for the flows a defense would actually want to purify.

6.3 System Limitations

The evaluation surfaced six limitations, collected here without their numbers (Section 5.9 carries the full analysis).

The first is a representational blind spot. On the `size` variants of the DNS, NTP, SSDP, and LDAP reflection floods, every model scores below chance against same-source benign flows: a manipulated flow resembles generic benign traffic more closely than that flood's own unmanipulated flows do. Three independent observations place the cause in the representation rather than in capacity, tuning, or training scope: models trained on each attack alone fail on exactly the same slices, the purifier reduces the perturbation footprint there without moving any verdict, and the deployed gate flags under 1% of their flows. Whatever distinguishes a padded single-response UDP flood from an unpadded one is not expressible in the 24 prefix statistics and 1088 header bits DEPT observes. Since these floods dominate the corpus by flow count, every aggregate in Chapter 5 is an average over near-perfect and below-chance sub-populations, not uniform competence.

The second is that purification without an adequate gate can subtract as well as add, as RQ2's discussion showed. The gate as built barely gates, since purifying every flow yields almost the same end-to-end result, and it answers arguably the wrong question: whether a flow is adversarial, not whether purification will improve its detectability.

The third belongs to the fusion design, and most directly contradicts the system's own premise: the meta-learner is beaten by its own Random Forest on the aggregate streaming axis, and by a wide margin on several sources (Section 5.5), where a stacker should be no worse than its best base learner. The carry-forward RF logit enters the fusion token as a single sparsely updated feature that attention under-weights against the dense TCN stream, and the per-attack comparison (Section 5.8.2) shows the pathology worsening even more so with less source diversity. What the fusion does buy, the packet-1 verdict and the robustness of RQ1, is real but narrower than the design hoped to achieve.

The fourth is methodological, but narrower than the previous ones. Purification gain and residual gap measured through DEPT's own TCN are circular, but the end-to-end claim does not rest on them: the deployment baseline (Section 5.8.1) scores clean, adversarial, and defended streams through the independent NIDS each attack targets. What remains is a specific weaker check: the defended full-stream figures run DEPT's models over some flows seen in training, so they do falter in this regard; each attack is judged by a single target NIDS rather than a panel of them; on `mirai` that target detects almost nothing even on the clean attack, leaving its rows uninformative; and Kitsune's partly volumetric detection rate is read only on full streams.

The fifth is scope, of two kinds. The purifier is attack-agnostic, correcting all 1090 dimensions without being told which the attacker moved, but the realization step overlays exactly the two quantities this threat model perturbs, packet size and inter-arrival time (Section 4.5.3). Against an adversary that

moved another header field, the feature-space correction would survive and the traffic-space one would not, and that flow would reach the protected NIDS unrepaired, flagged with nothing to hand back; the "attack-agnostic" label therefore belongs to detection and to the denoiser, not to the deployed pipeline as a whole. The feature space is bounded too: DEPT observes headers, timing, and direction, with no direct representation of payload. That was deliberate, since header and timing behaviour remain observable as encrypted traffic grows while payload inspection does not, but a manipulation expressed purely in application-layer content would be invisible, and payload may be exactly where the blind spot above lives.

The sixth is engineering. The tensorization front-end is pure Python, far slower than the native nPrint tool whose encoding it reimplements, and every neural stage runs a whole-flow forward pass, since no incremental or KV-cached inference path was implemented. Neither bound is conceptual, nPrint encoding being fixed-offset bit slicing the native tool performs at line rate, but as delivered they define the system's throughput floor (Section 6.4).

6.4 Future Work

The limitations above set most of the agenda. Both runs that earlier drafts deferred to this section are now delivered: the independent-NIDS validation of RQ2 (Section 5.8.1) and the direct gradient ablation for RQ1 (Section 5.7). What remains on the first is breadth, a panel of NIDS rather than each attack's single true target; what the second opened up is arguably more interesting than what it closed.

- **Is nPrint's encoding hiding the gradient?** The ablation found a gradient-guided attacker barely outperforming random search on the packet-size channel, and traced it to nPrint encoding size as 16 binary columns, so that adjacent byte counts sit far apart and the loss surface is jagged. This is uncomfortably close to gradient masking (Section 2.3.2); re-running the study against a model that consumes size as a scalar would separate the two, and the answer matters beyond just DEPT, but to anywhere nPrint-style bit encodings are used.
- **A stronger attacker for the forest.** The forest, having no gradient by construction, is only ever attacked by transfer. A gradient-based attack on a differentiable surrogate distilled from it would test whether its resistance is actually heterogeneity or merely distance from the TCN's gradient.
- **Latency and overhead evaluation.** Per-flow and per-packet inference latency, sustained throughput, and the overhead purification adds to the detect path are left to a following study, which would also settle whether the detect-then-purify cycle fits the real-time budget of Section 1.2.
- **A gate that predicts benefit rather than pure adversariality.** The end-to-end regressions above all follow from purifying flows that were still being detected. A gate trained instead on whether

purification raises or lowers the protected detector's score, a measure the system already computes for every test flow, would convert several reported losses into simple no-ops, leveling the end-to-end performance across all sources and modes.

- **An attack-agnostic realization step.** The purifier learns which dimensions to correct without being told; the overlay that returns its output to the wire is hand-written for two of them. Closing that gap means deriving the fields to edit from the residual Δ itself and reconciling them against the protocol's constraints, in the spirit of PANTS's SMT-checked perturbations [43], so that the defense re-emits traffic for manipulations it was never designed around. This would fully realize the attack-agnostic claim.

- **A fusion that never falls below its best base learner.** The meta-learner's failure to match its own Random Forest suggests re-emitting the RF evidence at every packet with an explicit staleness weight rather than in a single carry-forward slot, or gating the stacker so that it falls back to the stronger base opinion when the two diverge.

- **Finding what separates a padded flood from an unpadded one.** Narrowing down the below-chance results to the feature representation turns a failure into a specific question: which observable quantity, absent from the 24 prefix statistics and the 1088 header bits, distinguishes these flows? Answering it would repair the largest hole in the system's coverage.

- **Payload-aware representation.** Extending the feature space beyond headers and timing, where the traffic still permits it, would widen the class of manipulations the defense can observe.

- **A deployment-grade front-end.** Replacing the Python tensorization with a native implementation, like DPDK, and giving the models an incremental inference path, would let a deployment study test the pipeline against sustained line-rate traffic.

Beyond these, the direction this thesis considers most significant is **continuous adaptation of the defense**. A static defense, which is what DEPT is once trained, is a fixed target: an adversary that probes it long enough learns its blind spots and routes around it, as the arms-race framing of Chapter 3 makes clear. The natural next step is to close the loop, casting the defense as a Reinforcement Learning (RL) agent for which detection outcomes are the reward signal [74–76]. Framed as a Markov Decision Process (MDP), it would sit outside the per-packet critical path, observe the ensemble's verdicts and the attacker's drift, and retune the defense against an evolving generator such as TANTGen, its success measured across attacker generations rather than at a single point. Every component DEPT already has is a building block of that loop; what this thesis delivers is the static defense the adaptive one would start from.

In closing, this work argued that a defense for learning-based intrusion detection is only worth measuring against an attack that can exist on the wire, and it built and evaluated one under that principle end to end. Within its stated limits, DEPT detects a realistic black-box manipulation from the first packet and undoes it before the protected system consumes the flow; where it fails, it fails for reasons the evaluation identifies rather than hides. At the end of the day, it is a defense meant to be built upon, and RL seems the right place to leave it.

Bibliography

[1] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, pp. 16:1–16:99, 2018.

[2] S. Wang, J. F. Balarezo, S. Kandeepan, A. Al-Hourani, K. G. Chavez, and B. Rubinstein, "Machine learning in network anomaly detection: A survey," *IEEE Access*, vol. 9, pp. 152 379–152 396, 2021.

[3] Z. Xu, Y. Wu, S. Wang, J. Gao, T. Qiu, Z. Wang, H. Wan, and X. Zhao, "Deep learning-based intrusion detection systems: A survey," *arXiv preprint arXiv:2504.07839*, 2025.

[4] F. M. Anis, M. AlAbdullatif, S. Aljbli, and M. Hammoudeh, "A survey on the applications of deep learning in network intrusion detection systems to enhance network security," *IEEE Access*, vol. 13, pp. 185 357–185 373, 2025.

[5] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *2nd International Conference on Learning Representations (ICLR 2014)*, 2014.

[6] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[7] K. He, D. D. Kim, and M. R. Asghar, "Adversarial machine learning for network intrusion detection systems: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 538–566, 2023.

[8] C. Zhang, X. Costa-Pérez, and P. Patras, "Tiki-Taka: Attacking and defending deep learning-based intrusion detection systems," in *Proceedings of the 2020 ACM SIGSAC conference on cloud computing security workshop*, 2020, pp. 27–39.

[9] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing properties of adversarial ML attacks in the problem space," in *2020 IEEE symposium on security and privacy (SP)*. IEEE, 2020, pp. 1332–1349.

[10] M. J. Hashemi, G. Cusack, and E. Keller, "Towards evaluation of NIDSs in adversarial setting," in *Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks*, 2019, pp. 14–21.

[11] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, 2017, pp. 506–519.

[12] G. Apruzzese, P. Laskov, and J. Schneider, "SoK: Pragmatic assessment of machine learning for network intrusion detection," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 592–614.

[13] I. d. A. Pissarra, "TANTGen: Transformer-based adversarial network traffic generator," Master's thesis, Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal, 2026, unpublished Master's Thesis.

[14] D. H. Wolpert, "Stacked generalization," *Neural networks*, vol. 5, no. 2, pp. 241–259, 1992.

[15] P. Samangouei, M. Kabkab, and R. Chellappa, "Defense-GAN: Protecting classifiers against adversarial attacks using generative models," in *International Conference on Learning Representations (ICLR)*, 2018.

[16] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Network and Distributed System Security Symposium (NDSS)*, 2018.

[17] "Intrusion detection evaluation dataset (CIC-IDS2017)," [Online]. Available: https://www.unb.ca/cic/datasets/ids-2017.html, accessed: 2025-12-30.

[18] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, "Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy," in *2019 International Carnahan Conference on Security Technology (ICCST)*. IEEE, 2019, pp. 1–8.

[19] I. H. Sarker, "Deep cybersecurity: a comprehensive overview from neural network and deep learning perspective," *SN Computer Science*, vol. 2, no. 3, p. 154, 2021.

[20] A. Salim, O. Salim, O. M. Khudhur, S. M. Al-Barzinji, and F. M. Jasem, "Feature selection techniques in intrusion detection systems: A review," *Journal of Cybersecurity and Information Management*, vol. 17, no. 2, pp. 97–112, 2026.

[21] S. Ning, K. Nguyen, S. Bagchi, and Y. Park, “The study of feature engineering in machine learning and deep learning for network intrusion detection systems,” in *2024 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 2024, pp. 1–5.

[22] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, “New directions in automated traffic analysis,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2021, pp. 3366–3383.

[23] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, “End-to-end encrypted traffic classification with one-dimensional convolution neural networks,” in *2017 IEEE international conference on intelligence and security informatics (ISI)*. IEEE, 2017, pp. 43–48.

[24] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, “Deep packet: A novel approach for encrypted traffic classification using deep learning,” *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[25] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” in *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022, pp. 3971–3988.

[26] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[27] ——, “Bagging predictors,” *Machine learning*, vol. 24, no. 2, pp. 123–140, 1996.

[28] T. J. Lucas, I. S. De Figueiredo, C. A. C. Tojeiro, A. M. G. De Almeida, R. Scherer, J. R. F. Brega, J. P. Papa, and K. A. P. Da Costa, “A comprehensive survey on ensemble learning-based intrusion detection approaches in computer networks,” *IEEE Access*, vol. 11, pp. 122 638–122 676, 2023.

[29] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.

[30] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, “WaveNet: A generative model for raw audio,” *arXiv preprint arXiv:1609.03499*, 2016.

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[32] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, “ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification,” in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.

[33] J. Ghadermazi, S. Hore, A. Shah, and N. D. Bastian, "GTAE-IDS: Graph transformer-based autoencoder framework for real-time network intrusion detection," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 4026–4041, 2025.

[34] S. Ennaji, F. De Gaspari, D. Hitaj, A. Kbidi, and L. V. Mancini, "Adversarial challenges in network intrusion detection systems: Research insights and future prospects," *IEEE Access*, vol. 13, pp. 148 613–148 645, 2025.

[35] P. Sun, S. Li, J. Xie, H. Xu, Z. Cheng, and R. Yang, "GPMT: Generating practical malicious traffic based on adversarial attacks with little prior knowledge," *Computers & Security*, vol. 130, p. 103257, 2023.

[36] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *6th International Conference on Learning Representations (ICLR)*, 2018.

[37] N. Alhussien and A. Aleroud, "AdvPurRec: Strengthening network intrusion detection with diffusion model reconstruction against adversarial attacks," in *2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 2024, pp. 1638–1646.

[38] X. Li, W. Sun, H. Chen *et al.*, "ADBM: Adversarial diffusion bridge model for reliable adversarial purification," in *International Conference on Learning Representations (ICLR)*, 2025.

[39] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 582–597.

[40] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *International Conference on Machine Learning (ICML)*. PMLR, 2019, pp. 1310–1320.

[41] J. Yan, Z. Liu, Y. Xie, S. Liang, L. Liu, and K. Xu, "CertTA: Certified robustness made practical for learning-based traffic analysis," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025, pp. 7349–7368.

[42] E. Abad Rocamora, G. G. Chrysos, and V. Cevher, "Certified robustness under bounded Levenshtein distance," in *International Conference on Learning Representations (ICLR)*, 2025.

[43] M. Jin and M. Apostolaki, "Robustifying ML-powered network classifiers with PANTS," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 7291–7310.

[44] I. Bibers, O. Arreche, W. Alayed, and M. Abdallah, “Ensemble-IDS: An ensemble learning framework for enhancing AI-based network intrusion detection tasks,” *Applied Sciences*, vol. 15, no. 19, p. 10579, 2025.

[45] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of computer and system sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[46] X. Yang, Y. Zhao, and X. Chen, “A novel transformer-based stacking ensemble method with multi-model integration for cancer classification,” *PeerJ Computer Science*, vol. 11, p. e3314, 2025.

[47] O. Gungor, T. Rosing, and B. Aksanli, “STEWART: Stacking ensemble for white-box adversarial attacks towards more resilient data-driven predictive maintenance,” *Computers in Industry*, vol. 140, p. 103660, 2022.

[48] Z. Yang, L. Li, X. Xu, S. Zuo, Q. Chen, P. Zhou, B. I. P. Rubinstein, C. Zhang, and B. Li, “TRS: Transferability reduced ensemble via promoting gradient diversity and model smoothness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 17 642–17 655.

[49] B. Biggio, I. Corona, G. Fumera, G. Giacinto, and F. Roli, “Bagging classifiers for fighting poisoning attacks in adversarial classification tasks,” in *Multiple Classifier Systems: 10th International Workshop (MCS 2011)*. Springer, 2011, pp. 350–359.

[50] R. Chen, Z. Li, J. Li, J. Yan, and C. Wu, “On collective robustness of bagging against data poisoning,” in *International Conference on Machine Learning*. PMLR, 2022, pp. 3299–3319.

[51] A. Athalye, N. Carlini, and D. Wagner, “Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples,” in *International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 274–283.

[52] M. Hajizadeh, P. Golchin, E. Nowroozi, M. Rigaki, V. Valeros, S. García, M. Conti, and T. Bauschert, “DeepRed: A deep learning-powered command and control framework for multi-stage red teaming against ML-based network intrusion detection systems,” in *19th USENIX WOOT Conference on Offensive Technologies (WOOT 25)*, 2025, pp. 103–127.

[53] P. Kumar, A. S. M. Tayeen, S. Misra, H. Cao, J. Liu, Q. Gong, and J. Harikumar, “NetDiffuser: Deceiving DNN-based network attack detection systems with diffusion-generated adversarial traffic,” *arXiv preprint arXiv:2603.08901*, 2026.

[54] F. Croce and M. Hein, “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 2206–2216.

[55] M.-I. Nicolae, M. Sinn, M. N. Tran, B. Buesser, A. Rawat, M. Wistuba, V. Zantedeschi, N. Baracaldo, B. Chen, H. Ludwig, I. M. Molloy, and B. Edwards, "Adversarial robustness toolbox v1.0.0," arXiv, Tech. Rep. arXiv:1807.01069, 2018.

[56] N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. Goodfellow, A. Madry, and A. Kurakin, "On evaluating adversarial robustness," arXiv, Tech. Rep. arXiv:1902.06705, 2019.

[57] F. Tramèr, N. Carlini, W. Brendel, and A. Madry, "On adaptive attacks to adversarial example defenses," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020, pp. 1633–1645.

[58] B. I. Rubinstein, B. Nelson, L. Huang, A. D. Joseph, S.-h. Lau, S. Rao, N. Taft, and J. D. Tygar, "ANTIDOTE: understanding and defending against poisoning of anomaly detectors," in *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement*, 2009, pp. 1–14.

[59] G. Wan, Z. Shi, W. Huang, G. Zhang, D. Tao, and M. Ye, "Energy-based backdoor defense against federated graph learning," in *The Thirteenth International Conference on Learning Representations*, 2025.

[60] J. Xu, Z. Zhang, and R. Hu, "Detecting backdoor attacks in federated learning via direction alignment inspection," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 20 654–20 664.

[61] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A.-R. Sadeghi, "CrowdGuard: Federated backdoor detection in federated learning," in *Network and Distributed System Security Symposium (NDSS)*, 2024.

[62] R. Zhu, D. Tang, S. Tang, X. Wang, and H. Tang, "Selective amnesia: On efficient, high-fidelity and blind suppression of backdoor effects in trojaned machine learning models," in *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2023, pp. 682–700.

[63] K. Kumari, P. Rieger, H. Fereidooni, M. Jadliwala, and A.-R. Sadeghi, "BayBFed: Bayesian backdoor defense for federated learning," in *2023 IEEE symposium on security and privacy (SP)*. IEEE, 2023, pp. 737–754.

[64] D. T. Nguyen, N. N. Tran, T. T. Johnson, and K. Leach, "PBP: Post-training backdoor purification for malware classifiers," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[65] F. Alotaibi and S. Maffeis, "Mateen: Adaptive ensemble learning for network anomaly detection," in *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses*, 2024, pp. 215–234.

[66] M. Zhang, K. Bi, W. Chen, Q. Chen, J. Guo, and X. Cheng, "CausalDiff: Causality-inspired disentanglement via diffusion model for adversarial defense," *Advances in Neural Information Processing Systems*, vol. 37, pp. 124 791–124 819, 2024.

[67] Z. Li, H. Zhou, M. Rei, and L. Specia, "DiffuseDef: Improved robustness to adversarial attacks via iterative denoising," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 9259–9274.

[68] M. A. Merzouk, E. Beurier, R. Yaich, N. Boulahia-Cuppens, F. Cuppens, and F. Khomh, "Diffusion-based adversarial purification for intrusion detection," in *Data and Applications Security and Privacy XXXIX: 39th Annual IFIP WG 11.3 Conference (DBSec 2025)*. Springer, 2025, pp. 351–370.

[69] V. Kumar, K. Kumar, and M. Singh, "NIDS-CBAD: detecting adversarial attacks in network intrusion detection systems using domain constraints," *International Journal of Machine Learning and Cybernetics*, vol. 16, no. 7-8, pp. 4213–4234, 2025.

[70] O. Olukola and N. Rahimi, "AMDS: Attack-aware multi-stage defense system for network intrusion detection with two-stage adaptive weight learning," *arXiv preprint arXiv:2603.00859*, 2026.

[71] S. Ren, J. Jin, G. Niu, and Y. Liu, "ARCS: Adaptive reinforcement learning framework for automated cybersecurity incident response strategy optimization," *Applied Sciences*, vol. 15, no. 2, p. 951, 2025.

[72] G. Wilson, G. Goh, Y. Jiang, A. Gupta, J. Wang, D. Freeman, and F. Dinuzzo, "Predictive response optimization: Using reinforcement learning to fight online social network abuse," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3239–3256.

[73] W. Qiao, Y. Feng, T. Li, Z. Ma, Y. Shen, J. Ma, and Y. Liu, "Slot: Provenance-driven APT detection through graph reinforcement learning," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 963–977.

[74] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[75] T. T. Nguyen and V. J. Reddi, "Deep reinforcement learning for cyber security," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3779–3795, 2023.

[76] G. Caminero, M. Lopez-Martin, and B. Carro, "Adversarial environment reinforcement learning algorithm for intrusion detection," *Computer Networks*, vol. 159, pp. 96–109, 2019.